Ingegneria della conoscenza

Alessandro Carella

Indice

1. Ingegneria della conoscenza 5
   1.1. Sistemi intelligenti basati su Conoscenza 5
   1.2. Dimensioni dello spazio di progettazione 6
      1.2.1. Modularità 7
      1.2.2. Schema di Rappresentazione 7
   1.3. Progettazione si Sistemi Intelligenti 12
      1.3.1. Semplificazione di Ambienti e Sistemi 12
      1.3.2. Compiti e Problemi 12
      1.3.3. Rappresentazione interna 12
   1.4. Soluzioni 13
      1.4.1. Modelli: Livelli di Astrazione 13
   1.5. Sistemi Intelligenti basati su Conoscenza 14
   1.6. Domini applicativi 15
2. Ricerca di soluzioni in spazi di stati 15
   2.1. Risoluzione di problemi mediante ricerca 15
   2.2. Spazi di stati 16
   2.3. Ricerca su Grafo 18
   2.4. Algoritmo di ricerca generico 19
   2.5. Strategie di ricerca non informate 21
      2.5.1. Ricerca in ampiezza 21
      2.5.2. Ricerca in profondità 22
      2.5.3. Approfondimento iterativo 23
      2.5.4. Ricerca di soluzioni a costo minimo 24
   2.6. Ricerca Informata (Euristica) 25
      2.6.1. Euristica 25
      2.6.2. DFS Euristica e Greedy best-First Search 26
      2.6.3. Ricerca A^* 26
      2.6.4. Algoritmo Branch and Bound 28
      2.6.5. Progettazione della funzione Euristica 30
   2.7. Potatura dello Spazio di Ricerca 31
      2.7.1. Potatura dei Cicli 31
      2.7.2. Potatura di Percorsi Multipli 31
      2.7.3. Consuntiva sulle Strategie di Ricerca 34
   2.8. Strategia di Ricerca più sofisticate 34
      2.8.1. Direzione della ricerca 34
      2.8.2. Ricerca bidirezionale 34

2.8.3. Ricerca in una gerarchia di astrazioni 35
2.8.4. Uso della programmazione dinamica 35
3. Ragionamento con vincoli 36
3.1. Variabili e vincoli 36
3.1.1. Variabili e Assegnazioni 37
3.1.2. Vincoli 37
3.1.3. Problema di soddisfacimento di vincoli 38
3.2. Risoluzione di CSP tramite ricerca 39
3.2.1. Algoritmi su Grafo di Ricerca 39
3.3. Algoritmi basati su Consistenza 40
3.4. Separazione dei domini 41
3.5. Eliminazione di Variabili 43
3.6. Ricerca Locale 45
3.6.1. Miglioramento Iterativo 46
3.6.2. Algoritmi stocastici 47
3.7. Varianti della ricerca Locale 48
3.7.1. Tabu Search 48
3.7.2. Passo di Massimo Miglioramento 48
3.7.3. Scelta a due fasi 49
3.7.4. Algoritmo Any-Conflict 49
3.7.5. Simulated Annealing 49
3.7.6. Ripartenza casuale 51
3.8. Algoritmi basati su popolazioni 51
3.9. Beam Search 51
3.9.1. Beam Search Stocastica 52
3.9.2. Algoritmi Genetici 52
3.10. Ottimizzazione 54
3.10.1. Metodi Sistematici per il Caso Discreto 54
3.10.2. Ricerca locale per l'ottimizzazione 55
3.10.3. Domini Continui: Discesa di Gradiente 55
4. Rappresentazione e Ragionamentoi Proposizionale 56
4.1. Proposizioni 56
4.1.1. Sintassi del Calcolo Proposizionale 56
4.1.2. Semantica del Calcolo Proposizionale 57
4.1.3. Semantica: punto di vista del progettista 59
4.1.4. Semantica: Punto di vista della Macchina 59
4.2. Vincoli Proposizionali 60
4.2.1. Forma Clausale per CSP 60
4.3. Clausole Definite Proposizionali 61
4.3.1. Domande e Risposte 63

4.3.2. Dimostrazioni 64
4.3.3. Procedura Bottom-Up 64
4.3.4. Procedura Top-Down 66
4.4. Interrogare l'utente 70
4.5. Debugging a Livello di Conoscenza 71
4.5.1. Risposte Errate 71
4.5.2. Risposte mancanti 72
4.6. Dimostrazione per Contraddizione 72
4.6.1. Clausole di Horn 72
4.6.2. Assumibili e Conflitti 73
4.6.3. Diagnosi basata sulla consistenza 74
4.6.4. Ragionamento con Assunzioni e Clausole di Horn 75
4.7. Assunzione di Conoscenza Completa 77
4.7.1. Ragionamento Non Monotono 80
4.7.2. Procedure di Dimostrazione per la NAF 80
4.8. Abduzione 83

Ingegneria della conoscenza

Definizione 1 *(Ingegneria della conoscenza)* **.** L'Ingegneria della Conoscenza è il processo di acquisizione, strutturazione e rappresentazione delle competenze umane in un formato che le macchine possono comprendere e utilizzare. Può essere vista come l'arte e la scienza di costruire sistemi basati sulla conoscenza in grado di ragionare, prendere decisioni e risolvere problemi complessi in modi che rispecchiano il pensiero umano.

Nel contesto dell'*intelligenza artificiale* che sta rimodellando ogni settore, l'ingegneria della conoscenza rappresenta il ponte fondamentale tra le competenze umane e l'intelligenza artificiale. Gli ingegneri della conoscenza possono essere considerati come traduttori della conoscenza imp;licita che *vive* nella mente degli esperti: essi la convertono in formati espliciti e strutturati che le macchine possono elaborare e sulle quali possono agire.

1.1. Sistemi intelligenti basati su Conoscenza

L'ingegneria della conoscenza è una disciplina ingegneristica atta a progettare e sviluppare *sistemi intelligenti* basati su conoscenza *esplicita*.
L'intelligenta artificiale può essere definita come la disciplina che studia sintesi e analisi degli *agenti computazionali*, ossia dei sistemi che agiscono in modo *intelligente*. Essi possono svolgere azioni in un *ambiente* e vanno giudicati **solo** per tali azioni.
Aspetti salienti dell'*azione intelligente* sono i seguenti:

- l'appropriatezza rispetto alle circostanze, agli obiettivi, ai limiti;
- la considerazione delle conseguenza a breve/lungo termine delle azioni;
- l'apprendimento dall'esperienza;
- la flessibilità rispetto ai cambiamenti.

Un **agente computazionale** è un agente le cui decisioni/azioni siano spiegabili in termini di calcolo automatico. Esso è implementato su sistema *fisico* e presenta le seguenti *limitazioni*:

- non è onnisciente né onnipotente;
- le capacità di osservazione sono specializate e vincolate dal dominio applicativo;
- la sua memoria è finita e a tempo limitato.

La disciplina dell'AI può essere inquadrata sotto diverse prospettive:

- la prospettiva *scientifica* considera l'AI come displina finalizzata a comprendere i principi del comportamento intelligente:
  - analizzando sistemi naturali e artificiali;
  - formulando *teorie* sulla costruzione di agenti intelligenti che possano essere supportate da *implementazioni* per la loro *verifica sperimentale*;
- la prospettiva *ingegneristica* inquadra l'AI come lo studio di nuove tecnologie atte risolvere specifici problemi tipicamente risolti da agenti umani:
  - progetto e sintesi di *sistemi* intgelligenti *basati su conoscenza* (KBS) *testabili*: qualità valutabili attraverso metrodi standard dell'ing. informatica;
  - applicazioni costruite per la loro *utilità* in un dato dominio.

Altri obiettivi dell'AI:

- **intelligence augmentation**: accrescimento dell'intelligenza e della creatività umane. Alcuni esempi:
  - sistema diagnostico: aiuta i medici a prendere decisioni migliori;
  - motore di ricerca: aiuta a superare i limiti della memoria umana;
  - traduzione automatica: aiuta la gente a comunicare.
- **integrazione** dei compiti di agenti umani e sistemi intelligenti che collaborano alla soluzione di problemi; si parla di modalità **human-in-the-loop**.

C'è differenza tra la definizione di intelligenza *naturale* e l'intelligenza *artificiale*. L'intelligenza *naturale* emerge spontanaemente in natura mentre, quella *artificiale*, viene progettata per specifici scopi.
L'ipotesi fondamentale può essere riassunta con la seguente formula:

$$\text{ragionamento} \approx \text{computazione}$$

Tale ipotesi è collegata alla *tesi di Church-Turing*: esiste un livello di astrazione nel quale il ragionamento coincide con la manipolazione di simboli.
In termini generali potremmo definire la **conoscenza** un'informazione stabile, non transitoria.

1.2. Dimensioni dello spazio di progettazione

La *complessità* dei sistemi intelligenti varia e dipende dalle finalità e dai diversi contesti in cui sono immersi. Ciò vale dai casi semplici a quelli più complessi.
Le **dimensioni della complessità** nella progettazione:

- definiscono uno *spazio di progettazione*: si hanno diversi sistemi a seconda delle loro tipologie;
- forniscono forme di *decomposizione sommaria* dello spazio, cui andranno aggiunte altre scelte da fare;
- interagiscono fra loro: dimensioni combinabili anche se studiate separatamente.

Dimensione	**Valori**
Modularità	piatta | modulare | gerarchica
Orizzonte della pianificazione	nessuna pianificazione | #tappe finito (miopia) | indefinito | infinito (processo)
Rappresentazione del mondo	stati | (tuple di) feature | individui + relazioni (binarie) #mondi: n_s | 2^{n_f} | $2^{n_i^2} \cdot n_r$ (n_s = #stati, n_f = #feature, n_i = #individui e n_r = #relazioni)
Incertezza	*osservabilità del mondo:* totale | parziale *effetto:* deterministico | stocastico
Preferenze	obiettivi (da raggiungere | mantenere) | compromessi (preferenze cardinali | ordinali)
Apprendimento	*conoscenza data / appresa*
Limiti delle risorse	*razionalità perfetta / limitata*
Numero di attori	singolo | multipli agenti (distribuiti, cooperativi | competitivi)
Interattività	*ragionamento (computazione) online / offline*

1.2.1. Modularità

Il tipo di **modularità** indica il grado di decomposizione di un sistema in *moduli* interagenti da prendere in considerazione separatamente. Ciò serve a *dominare la complessità*, nei sistemi software ma anche nelle organizzazioni. Tipicamente la modularità viene espressa attraverso una decomposizione gerarchica: ogni modulo è organizzato in sotto-mopduli a loro volta organizzati gerarchicamente fino al livello base; astrazione procedurale e OOP servono a sfruttare modularità e astrazione.
Le strutture possibili sono: *piatta*, ossia senza struttura organizzativa; *modulare*, se il sistema viene decomposto in moduli interagenti considerabili separatamente; *gerarchica* quando nel sistema i moduli possono essere decomposti in sotto-moduli interagenti, a loro volta decomponibili.
Le modalità di *ragionamento* dipendono dal tipo di struttura: con uno struttura piatta o modularesi ha un singolo livello di astrazione; con una gerarchica si hanno multipli livelli di astrazione.

1.2.2. Schema di Rappresentazione

Lo **schema di rappresentazione** riguarda la *descrizione del mondo*. Le rappresentazioni alternative possibili sono: a *stati*, con *caratteristiche / proposizioni* oppure con individui e *relazioni*.

Stati

Gli **stati** distinti del mondo hanno un impatto sul comportamento del sistema. Essi sono fattorizzabili in stati *interni* e stati dell'*ambiente*.

> **Esempio 1.** Un termostato può essere modellato usando 6 stati:
> - stati *interni*: **spento**, **riscaldamento**,
> - stati *ambiente*: **freddo**, **confortevole**, **caldo**;
>   - ambiente freddo: passare o restare in modalità riscaldamento;
>   - ambiente molto caldo: può passare nello stato spento;
>   - ambiente confortevole: rimanere nello stato corrente;
> - *azioni*: scaldare nello stato riscaldamento, altrimenti passare nello stato spento.

Caratteristiche

Il ragionamento si basa su **caratteristiche**, dette *feature*, degli stati o *proposizioni* booleane invece della loro enumerazione. Uno stato può essere descritto in termini di caratteristiche, con un valore per ogni stato.

> **Esempio 2** *(Domotica)* **.** In un sistema per la domotica le caratteristiche potrebbero essere:
> - posizione degli interruttori;
> - stato di ogni interruttore (*in_funzione, in_corto, fuori_uso*);
> - stato dei punti luce, ad esempio *pos_s2* con valore *up* se l'interruttore *s2* è accesso e *down* se spento;
> - stato della casa descritto in termini dei valori di ciascuna delle caratteristiche.

Proposizioni

Una **proposizione** è una caratteristica *booleana*. Risulta una rappresentazione *compatta*: sono state comprese *regolarità* importanti sul dominio. Ad esempio con 30 proposizioni codificano $2^{30} \approx 10^9$ stati quindi è più facile specfiicare e ragione con 30 proposizioni che con oltre un miliardo di stati.

Relazioni e individui

Nella descrizione di mondi complessi una relazione su un singolo individuo è una **proprietà**. Si può definire una *caratteristica* per ogni possibile relazione tra individui, come ad esempio $R(a, b)$, che potrebbe corrispondere a «Nico studia Informatica», ossia $studia(Nico, Informatica)$. Le *descrizioni relazionali* sono più convenienti di caratteristiche e proposizioni:

- ad esempio, con una sola relazione binaria e 100 indvidui si possono rappresentare $100^2 = 10000$ proposizioni e 2^{10000} stati;
- si possono considerare intere classi di individui senza enumerarne caratteristiche/ proposizioni, o addirittura i numerossisimi stati;

- permettono di ragione su infiniti individui; ad esempio l'insieme dei numeri, o l'insieme di tutte le stringhe, cosa impossibile in termini di stati.

Incertezza

Si progetta anche tenendo conto dell'**incertezza** insita nel dominio considerato:

1. percezione / osservazione;
2. effetti delle decisioni / azioni.

Osservazione incerta

A volte è possibile l'*osservazione diretta* dello stato del mondo. Più spesso accade che la percezione dello stato sia *difettosa* o *parziale / indiretta*: al più si può disporre di una *distribuzione* di probabilità sull'insieme degli stati possibili su quanto si osserva.

L'**incertezza sulla percezione** riguarda la possibilità di determinare lo stato del mondo attraverso osservazioni. Uno stato può essere:

- *pienamente osservabile*: si può riconoscere dalle osservazioni, assunzione spesso fatta per ragioni di trattabilità dei problemi;
- *parzialmente osservabile*: non osservato direttamente; è possibile che più stati portino alle stesse osservazioni oppure le osservazioni sono *rumorose*.

Effetto incerto

In certi casi è possibile conoscere sempre l'effetto delle azioni/decisioni: dato uno stato e un'azione/decisione, si può predire *precisamente* lo stato risultante dall'applicazione dell'azione. A volte è difficile fare tale previsioni: al più si avere una *distrubuzione di probabilità* sugli effetti possibili.

L'*incertezza sugli effetti* prevede che la loro *dinamica* possa essere:

- *deterministica* e lo stato risultante viene determinato esattamente dall'azione e dallo stato precedente;
- *aleatoria*, con una probabilità sui possibili stati risultanti: ha senso solo se il mondo è completamente osservabile.

Preferenza

Gli agenti sono spesso *utilitaristici*: la scelta di un'azione è dettata dai risultati attesi più desiderabili; ci possono essere le finalità *semplici* o anche preferenze *complesse*.

Le **preferenze** si caratterizzano come:

- *finalità*, da raggiungere, detto *achievement goal*, o di conservazione, detto *mainenance goal*, in ogni stato visitato;
- *preferenze complesse*: sono compromessi sul vantaggio derivante dei vari risultati, eventualemten anche in momenti diversi. Si distinguono:
  - preferenze *ordinali*: conta solo l'ordine;
  - preferenze *cardinali*: conta anche la grandezza del valore.

Apprendimento

Non sempre il progettista dispone di un buon modello del sistema e del suoi ambiente: si devono usare dati da esperienze passate e altre sorgenti di conoscenza per migliorare il modello e prendere migliori decisioni.

La dimensione dell'**apprendimento** determina se la conoscenza sia data oppure se vada appresa.

Apprendere significa trovare il modello migliore che si adatti ai dati e si possono presentare diversi casi:

- caso *semplice*: regolare un insieme fisso di *parametri*;
- caso più difficile: scegliere preliminarmente la migliore *rappresentazione*, ad esempio feature o relazioni.

Si possono presentare diverse problematiche:

- utilizzo di *conoscenza di fondo*;
- *selezione* dei dati da raccogliere;
- *rappresentazione* dei dati e dei modelli;
- selezione dei *learning bias* appropriati;
- uso della conoscenza appresa per modificare decisioni / azioni dell'agente.

Limitatezza delle Risorse Computazionali

Le **limitazioni** sulle risorse computazionali disponibili spesso impediscono di prendere le migliori decisioni. Si fanno compromessi quindi sulla qualità della soluzione da cercare spesso necessari: spesso è preferibile una soluzioner ragionevole ma rapida che una migliore calcolata quando è troppo tardi perchè il mondo esterno nel frattempo è cambiato.

Tale dimensione determina se il sistema ragione per prender la migliore decisione: senza tener conto dei limiti, *razionalità perfetta*, oppure tenenendo conto dei limiti, *razionalità limitata*.

I *limiti* riguardano il *tempo*, la *memoria* e la *precisione*, il grado di approssimazione.

Un **algoritmo anytime** produce soluzioni che migliorano con il tempo:

- in qualunque momento produce la *miglior soluzione corrente*;
- si assicura che la qualità non decresca, si conserva la migliore soluzione trovata da restituire su richiesta;
- l'attesa può avere un *costo*: a volte meglio decidere/agire subito anzichè di aspettare una soluzione probabilemente migliore.

Orizzonte

L'**orizzonte** misura quanto lontano sia prevista la pianificazione del lavoro, ossia quanto in avanti ci si spinga a considerare le conseguenze delle azioni. Si possono distinguere sistemi/agenti software:

- *senza pianificazione*: nelle decisioni non considerano il futuro e il fattore-tempo non è preso in considerazione;

- a *orizzonte finito*: interessano solo un numero prefissato di passi;
- a *orizzonte indefinito*: numero di passi finito, ma non predeterminato;
- a *orizzonte infinito*: sempre attivo.

Numero di agenti

Difficoltà aggiuntive degli ambienti con altri agenti / sistemi sono:

- occorre ragionare sugli altri agenti secondo *strategie*: potrebbero cercare di confondere e manipolare o potrebbero cooperare; conviene agire in modo casuale se gli altri adottano strategie deterministiche;
- anche quando si cooperi e vi sia un file comune, il problema della *coordinazione* e della *comunicazione* rende il ragionamento multi-agente più complesso.

Dal punto di vista del singolo agente, la dimensione del **numero di agenti** prevede:

- ragionamento da *agente singolo*: gli altri come parte dell'ambiente;
- ragionevole se non ci sono altri agenti o se gli altri non cambieranno il comportamento in base alle sue azioni;
- ragionamento *multi-agente*: si prende in considerazione il ragionamento altrui;
- in caso di agenti intelligenti con *fini / preferenze* che dipendano da quello dalla *comunicazione* con gli altri;
- più difficile se gli agenti possono agire *simultaneamente* o se l'ambiente è solo *parzialmente osservabile*.

Interazione delle Dimensioni

Non si possono studiare le dimensioni separatamente perchè sono soggette a *interazioni complesse*:

- *Rappresentazione e modularità*: avendo una gerarchia di moduli semplici, il ragionamento può essere svolto su insieme finito di stati; altri livelli di astrazione richiedono il ragionamento su relazioni.
- *Orizzonte e modularità*: ad esempio, un cane robotico può ricevere una ricompensa quando risponde al richiamo; quando decide sui singoli movimenti, il momento del premio potrebbe essere lontano quindi l'orizzonte risulta potenzialmente indefinito.
- *Incertezza sull'osservazione e complessità del ragionamento*: molto più facile ragionare quando si conosce lo stato del mondo; l'incertezza su individui e relazioni è più complicata da trattare.
- *Incertezza sugli effetti e modularità*: ad un dato livello della gerarchiua, una decisione può essere *deterministica* mentre ad un altro potrebbe essere *stocastica*.
- *Molteplicità e modularità*: agente progettato attraverso più sistemi che interagiscono condvidendo il *fine comune* di rendere intelligente il comportamente dell'agente al livello superiore.
- *Apprendimento e rappresentazione*: spesso l'apprendimento lavora sulla rappresentazione di feature: determina modelli che portano a miglioir predizioni dei

valori di una specifica feature-obiettivi; ma si può lavorare anche su individui e relazioni, ovveroi l'apprendimento di gerarchie di modelli, in domini parzialmente osservabili e attraverso sistemi multipli.

- *Modularità e razionalità limitata* per un ragionamento più efficiente; il formalismo può diventare più complicato ma per costruire sistemi complessi servono: la decomposizione in componenti più piccole e delle approssimazioni per poter decidere in tempo accettabili anche in regime di memoria limitata.

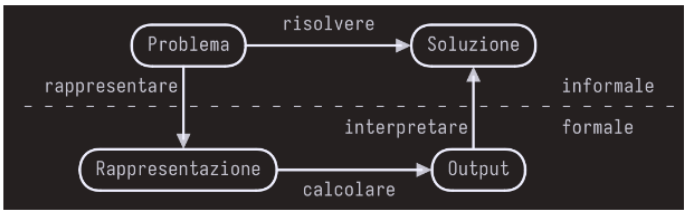


Figura 1.1: Risoluzione dei problemi

Figura 1: Risoluzione dei problemi

1.3. Progettazione si Sistemi Intelligenti

1.3.1. Semplificazione di Ambienti e Sistemi

La conoscenza del KBS non corrisponde a quella nella mente dell'esperto e/o del progettista.
Vi sono due casi-limite possibili:

- *sistema specializzato* nel suo dominio/task, poco utile fuori dal suo contest;
- *sistema flessibile* che si adatta ai contesti e accetta nuovi task.

Le strategie di costruzioni alternative sono:

- creazione di un modello dell'ambiente *semplificato* utile a costruire sistemi di ragionemento *complessi*
- sistemi *semplici* per contesti *complessi*.

1.3.2. Compiti e Problemi

In AI è importante **cosa** vada fatto/calcolato e non **come** ridotto a ricerca nello spazio di soluzioni possibili. Una descrizione di un **problema** dev'essere disponibile a livello informale.

1.3.3. Rappresentazione interna

La **conoscenza** può essere definita come informazione *stabile*, a *lungo termine* su un dominio, a differenza delle **credenze** sull'ambiente che sono più *transitorie*. Per poterci ragionare essa va rappresentata formalizzandola in termini di un **linguaggio di rappresentazione**. Una **base di conoscenza** risulterà come rappresentazione

interna al sistema, codificata attraverso idonee strutture dati.
Le *proprietà* principali degli schemi di rappresntazione sono le seguenti:

- *richezza espressiva* sufficiente alla risoluzione del problema;
- *vicinanza* ai termini naturali del problema; esiste una relazione fra il dominio e la sua rappresentazione il che permette verifiche di correttezza; la vicinanza è anche correlata alla *spiegabilità* dei meccanismi di ragionemento interni;
- *trattabilità*, propensione a un'elaborazione efficiente;
- *acquisibilità* a partire da dati e/o da esperienza pregressa da parte degli utenti.

1.4. Soluzioni

Come nell'ingegneria del software, dato un problema, il progettista deve definire cosa costituisca una **soluzione**. Le soluzioni possono essere di diverso tipo:

- *ottimali*: la migliore secondo una *misura di qualità*:
  - *ordinale*, tipica;
  - *cardinale*, se contano anche le grandezze relative;
  - combinazione di criteri multipli.
- *soddisfacente*: buona secondo una specifica delle risposte *adeguate*;
- *approssimata*: soluzione di qualità *prossima* a quella ottimale, sulla base di una misura *cardinale*; conveniente per ragioni di efficienza anche in virtù degli algoritmi adottati;
- *probabile*: *verosimilimente* una soluzione, con un determinato grado di certezza:
  - forma di approssimazione precisa di soluzione soddisfacente;
  - si possono distinguere i tassi di errori di *falsi-positivi* o di *falsi-negativi*.

1.4.1. Modelli: Livelli di Astrazione

Un **modello** è una rappresentazione del mondo che può essere statica, ciò che si crede vero, ovvero dinamica, il suo *funzionamento*. È utile lavorare a vari lvielli di dettaglio, ossia di **astrazione**: si rappresenta la parte del mondo di interesse trascurando i dettagli inutili. Il **livello di astrazione** determina un *ordine parziale*. Sono spesso ammissibili *più modelli* a diversi livelli di astrazione, anche in contraddizione tra loro: andranno giudicato in base alla loro utilità che per la loro correttezza.
Livelli d'astrazione comuni fra sistemi *biologici* e *computazionali* sono i seguenti:

- **livello della conoscenza** sul mondo esterno: considera quello che il sistema assume di sapere e i suoi obiettivi, ma non come ragiona, ossia non come calcolare la soluzione;
- **livello simbolico** interno: descrizione del modo di ragionare del sistema utile a implementare il livello precedente, manipolando simboli per produrre risposte.

1.5. Sistemi Intelligenti basati su Conoscenza

Un **agente intelligente** è un modello capace di percezione - ragionamento - aszione che si trova immerso in un **ambiente**. Un possibile schema è il seguente:

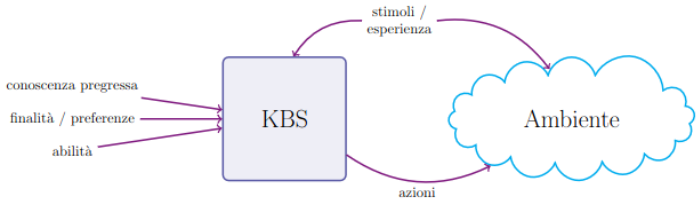


Figura 1.2: Schema di KBS / agente

Figura 2: Schema di un KBS

Il comportamento del KBS si basa su:

- **conoscenza pregressa** riguardo l'agente e il suo ambiente;
- **storia** dell'interazione con l'ambiente: **stimoli** ed **esperienza passata**;
- **obiettivi** da raggiungere o **preferenze** sugli stati del mondo;
- **abilità**, azioni primitivi di cui è capace.

Nella scatola nera si rappresenta un suo *stato interno delle credenze*, o **belief state**, comprendente la rappresentazione delle cose che si ritengono vere circa l'ambiente, cosa si è imparato, obiettivi intermedi presenti e futuri.
Il funzionamento di un KBS intelligente può essere schematizzato in due fasi, come nella seguente figura:

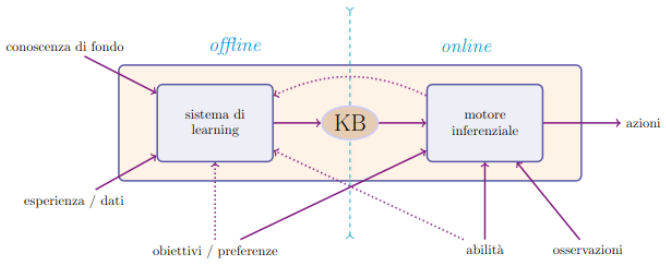


Figura 3: Funzionamento di un KBS

La KB viene coinvolta in momenti diversi:

- *Offline*, viene costruita/integrata usando conoscenza pregressa ed esperienze passate: spesso servono molti dati e conoscenza generale. Tale attività nei sistemi esperti non è automatizzata;
- *Onliune*, si prendono decisioni/compiono azioni e si aggiorna la KB; nel ragionamento si usano KB, osservazioni, obiettivi e abilità del sistema.

Una KB è utile nel futuro come *long-term memory*: viene appresa dai dati e dall'esperienza pregressa; un belief state, *short-term memory*, è un modello dell'ambiente attuale necessario tra successivi intervalli di tempo.
Uno schema di un KBS dettagliato è il seguente:

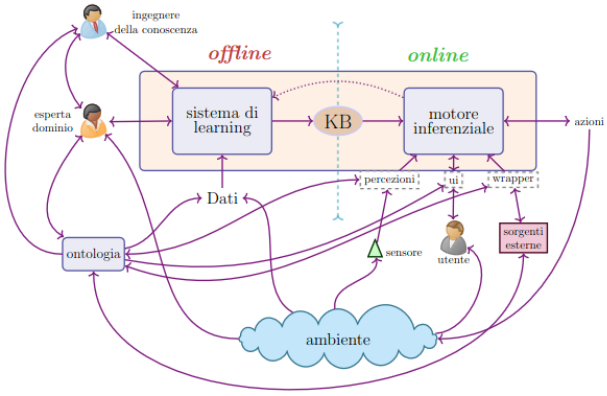


Figura 4: Schema dettagliato di un KBS

1.6. Domini applicativi

Tipici domini applicativi sono i seguenti:

- *robotica*;
- *diagnostica*: *assistente* che aiuta a capire i problemi di altri sistemi software e suggerisce rimedi;
- *tutoring*: *sistema* di aiuta agli studenti che interagisce fornendo informazioni su un argomento di interesse, assegnando compiti/test e valutando le capacità; deve comprendere la materia, lo studente e come questo impari;
- *e-commerce*: assistente agli acquisti di beni e servizi per conto dell'utente che sa riconoscere richieste e preferenze; sa trovare compromessi tra obiettivi distinti.

Ricerca di soluzioni in spazi di stati

2.1. Risoluzione di problemi mediante ricerca

La risoluzione di diversi problemi spesso consiste nella *ricerca* in uno specifico modello del mondo. Considernado il caso di un sistema che, dato un *obiettivo* da raggiungere, ragiona su un *modello* del mondo fatto di *stati*, in *assenza di incertezza*.

Per raggiungere questo scopo, si adotta una rappresentazione *piatta*, ovvero relativa a un singolo livello in una gerarchia.
L'*astrazione* del problema prevede la ricerca di un *percorso* che vada da un nodo di partenza a uno dei nodi-obiettivo, detto *goal*, in un *grafo orientato*.

> **Esempio 3** *(Navigatore)* **.** L'obiettivo è la ricerca del *miglior percorso* da un luogo a un altro: il più corto; quello di minimo costo; il più veloce; il più attrattivo, ecc.
> Ogni *stato* include informazioni su: localizzazione, direzione, velocità, mezzo di trasporto utilizzato e così via.

Una strategia di ricerca comune a molti problemi in AI è la seguente: il sistema computa su una *rappresentazione interna*. Il sistema riceve solo una descrizione di *cosa* rappresenti una soluzione; *come* ottenerla sarà il compito di un algoritmo di ricerca. Spesso questi problemi e i relativi algoritmi risolutivi sono NP-completi.
Tali problemi risultano *difficili* anche per agenti umani. Spesso non esistono soluzioni ottimali per cui ci si accontenta di soluzioni *soddisfacenti* in mancanza d'una *struttura* mappabile su caratteristiche del mondo fisico.

2.2. Spazi di stati

Possiamo formulare la nozione di azione intelligente in termini di uno **spazio di stati**. Uno *stato* contiene tutte le informazioni necesserie per predirre gli effetti di un'azione e per determinare se soddisfi l'obiettivo. Una ricerca su uno spazio di stati assume le seguenti nozioni:

- Gli agenti hanno una conoscenza perfetta dello spazio di stati.
- In qualsiasi momento, conosce in quale stato si trova; il mondo è quindi pienamente osservabile.
- L'agente ha un insieme di azioni con effetti deterministici conosciuti.
- L'agente ha un obiettivo da raggiungere e può determinare se uno stato soddisfa l'obiettivo.

Una **soluzione** di un problema di ricerca è una sequenza di azioni che porterà l'agente dallo stato corrente allo stato obiettivo.
Un **problema di ricerca su uno spazio di stati** comprende:

- un insieme di stati
- uno stato distinto detto *stato di partenza*
- per ogni stato, un insieme di azioni disponibili all'agente che si trova in tale stato
- una **funzione di azione** che, dato uno stato corrente e un'azione *(s, a)*, ritorna un nuovo stato in cui si transita per effetto dell'azione
- un **goal** che è una funzione booleana, *goal(s)*, che ritornerà *true* se s è uno stato che soddisfa l'obiettivo e, in tal caso, s è lo **stato obiettivo**

- un criterio che specifica la qualità di una soluzione accettabile; una **soluzione ottimale** massimizza tale criterio, ad esempio se ammette *qualsiasi* sequenza di azioni che porti a uno stato-obiettivo; ad esempio associando *costi* alle diverse azioni da cui discende un criterio del *minimo costo totale* delle azioni; a volte, sono soddisfacenti anche soluzioni sub-ottimali, ad esempio quelle con un costo aggiuntivo del +10% rispetto a quello di una soluzione ottimale.

Esempio 4 *(Robot-consegne)* **.** Un robot che debba effettuare consegne in un edificio dovrà essere in grado di risolvere *problemi di ricerca* di percorsi da un luogo ad un altro in un dato piano.
Occorrerà definire:
- gli *stati*: posizioni distinte;
- le *azioni*: spostamenti da un luogo a un altro nelle vicinanze;
- il *problema* specifico: portarsi da a (unico stato-obiettivo);
- la *soluzione*: sequenza di spostamenti.

Possibili *estensioni* dei problemi di ricerca riguardano:
- la possibilità di sfruttare una *struttura interna* agli stati: ad esempio nei videogame;
- il caso di stati *non* completamente *osservabili*: ad esempio il robot-consegne non conosce la posizione iniziale delle consegne, oppure un sistema di tutoring può non conoscere bene le attitudini di uno studente;
- l'aggiunta di azioni *stocastiche*: ad esempio possibilità di commettere errori oppure mancato apprendimento da parte dello studente di un argomento;
- la possibilità di definire, invece degli stati finali, *preferenze aggiuntive* complesse, in termini di *ricompense* o *punizioni*.

Esempio 5 *(Videogame)* **.** Si consideri lo spazio nella seguente figura.

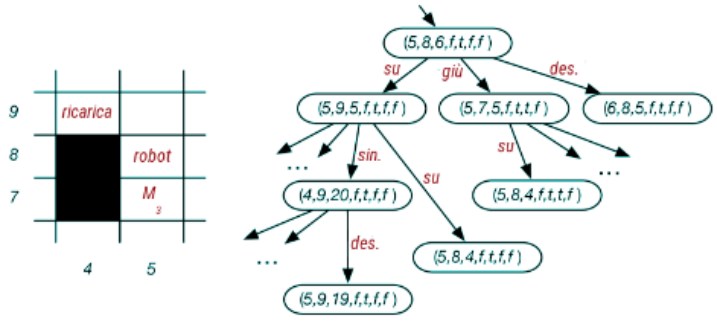


Figura 5: Spazio di stati del problema.

In tal caso definiamo:

- il *modello*: griglia nella quale ci si può muovere di una casella nelle 4 direzioni se non si è bloccati da un muro (caselle scure);
- l'*obiettivo*: raccogliere 4 monete $M_1, ..., M_4$, ognuna in una posizione iniziale nota, ad esempio C_3 in (5, 7);
- il *costo*: un'unità di energia per ogni passo: nessuna mossa è possibile senza energia; ricarica (+16 unità) presso una casella *ricarica*, ad esempio in (4, 9);
- lo *stato*: dato da una posizione x, y, dalle unità di carburante u possedute e da 4 flag c_i che indicano il possesso della relativa moneta: $(x, y, u, m_1, m_2, m_3, m_4)$;
- possibili stato-obiettivo $(5, 7, ?, t, t, t, t)$ dove ? indica un quantitativo qualsiasi di carburante.

2.3. Ricerca su Grafo

Per risolvere il problema, si definisce lo spazio di ricerca e si applica un algoritmo di ricerca. Come detto precedentemente, molti compiti possono essere ricondotti alla *ricerca* di percorsi in un grafo. Un modello *astratto* della soluzione sarà *indipendente* dal particolare dominio applicativo di interesse. Dato un *grafo orientato* fatto di nodi connessi da archi, si dovrà trovare un *percorso* da un nodo di partenza a uno boiettivo. Ci possono essere più modi per rappresentare uno stesso problema.

Un **grafo orientato** *esplicito* consiste di:

- un insieme di *nodi N*, anche non finito;
- un insieme di *archi A*, ossia di coppie *ordinate* di nodi, anche non infinito;
  - l'arco n_1, n_2 si dirà **uscente** da n_1 ed **entrante** in n_2
  - si dirà che il nodo n_2 è un **vicino** del nodo n_1 se e solo se $\exists \langle n_1, n_2 \rangle \in A$
- un **percorso** dal nodo s al nodo g, denotato da $\langle n_0, n_1, ..., n_k \rangle$, è una *sequenza* di nodi tale che se $s = n_0, g = n_l$ e $\forall i = 1, ..., k : \langle n_{i-1}, n_i \rangle \in A$; in alternativa, si può indicare una *sequenza di archi* $\langle n_0, n_1 \rangle, \langle n_1, n_2 \rangle, ..., \langle n_{k-1}, n_k \rangle$ o anche *sequenza di etichette* su tali archi: $\langle n_0, ..., n_i \rangle$ *parte iniziale* di $\langle n_0, n_1, ..., n_k \rangle$, con $i \leq k$.

Nel grafo esiste un insieme di *nodi-obiettivo* che si possono identificare anche tramite il predicato *goal(·)*, funzione booleana definita sui nodi. Una **soluzione** è un percorso dal nodo di partenza a uno obiettivo.
In alcuni casi è associato un **costo** a ogni arco: dato $\langle n_i, n_j \rangle$

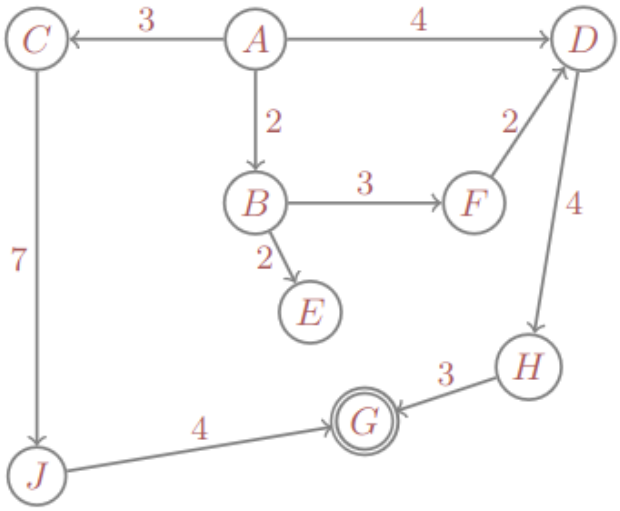
$$cost\big(\langle n_i, n_j \rangle\big) \in [0, +\infty[$$

estendibile anche ai percosi: dato $p = \langle n_0, n_1, ..., n_k \rangle$

$$cost(p) = \sum_{i=1}^{k} cost(\langle n_{i-1}, n_i \rangle)$$

Una soluzione **ottimale** p avrà costo minimo: $\nexists p'$ soluzione con $cost(p') < cost(p)$.

Esempio 6 *(Robot consegne, continuo.)* **.** Ricerca di un percorso da A a G nel mondo rappresentato nella seguente figura.



- $N = \{A, B, C, D, E, F, G, H, J\}$
- $A = \{\langle A, B\rangle, \langle A, C\rangle, \langle A, D\rangle, \langle B, E\rangle\langle B, F\rangle, \langle C, J\rangle, ...\}$ dove: E non ha vicini; C ha come vicino solo J; A ha i vicini B, C e D;
- vi sono 3 percorsi da A a G: $\langle A, D, H, G\rangle, \langle A, C, J, G\rangle, \langle A, B, F, D, H, G\rangle$; se A fosse nodo di partenza e G il nodo-obiettivo, ognuno di tali percorsi sarebbe una soluzione.

Un **ciclo** è un percorso non vuoto in cui primo e ultimo nodo coincidono: $\langle n_0, n_1, ..., n_k\rangle$ tale che $k > 0$ e $n_0 = n_k$.

Un **grafo aciclico orientato** (*directed acyclic graph*, DAG) è un grafo orientato senza cicli. Un **albero** è un DAG con un solo nodo, la **radice**, senza archi entranti. I nodi senza archi uscenti sono le sue **foglie**.

2.4. Algoritmo di ricerca generico

Un algoritmo di ricerca generico è un algoritmo indipendente dalla strategia di ricercadal grafo. Dato un grafo, si esplorano *incrementalmente* percorsi dai nodi di partenza verso nodi-obiettivo.

Si una una struttura dati per rappresentare la **frontiera**, detta *fringe*, dei percorsi già esplorati. Inizialmente la frontiera contiene percorsi costituiti dai soli *nodi di partenza*. Successivamente si effettua l'*espanzione* dei percorsi nella frontiera verso nodi inesplorati, fino a incontrare un nodo-obiettivo:

- si seleziona un percorso (rimuovendolo dalla frontiera);
- si estende il percorso con ogni arco uscente dall'ultimo nodo;
- si aggiungono alla frontiera i percorsi ottenuti.

ALGORITMO 1 — *Algoritmo di ricerca generico*

```
1:  procedure Search(G, s, goal)
2:  Input
3:      G: grafo con insiemi di nodi N e di archi A
4:      s: nodo di partenza
5:      goal: funzione booleana sui nodi
6:  Output
7:      percorso da s a un nodo per il quale goal sia vera
8:      oppure ⊥ quando non ci sono percorsi / soluzioni
9:  Local
10:     frontiera: insieme di percorsi
11: frontiera ← {⟨s⟩}
12: while frontiera ≠ ∅ do
13:     selezionare (e rimuovere) ⟨n_0, ..., n_k⟩ da frontiera
14:     if goal(n_k) then
15:         return ⟨n_0, ..., n_k⟩
16:     frontiera ← frontiera ∪ {⟨n_0, ..., n_k, n⟩ : ⟨n_k, n⟩ ∈ A}
17: return ⊥
```

Come descritto nell'Algoritmo di ricerca generico, l'inizializzazione parte dal nodo s.

La selezione *non è deterministica*, ha solo impatto sull'efficienza quindi una data strategia di selezione determina il percorso da scegliere. Il annidato può essere interpretato come *temporaneo*, continuando si trovano strade alternative; $\perp$ indica che non vi sono soluzioni; la condizione *goal(n_k)* viene testata doo la selezione della frontiera e non all'aggiunta del nuovo nood:

1. a volte esiste un *arco* vers un nodo-obiettivo ma di *costo elevato*: non sempre conviene terminare restituendo subito il percorso trovato in quanto potrebbe esserci un percorso di costo inferiore;
2. va tenuto conto del fatto che lo stesso test *goal()* può essere *costoso*; un percorso che termini con un nodo che non sia un nodo-obiettivo e che non abbia vicini andrebbe rimosso.

2.5. Strategie di ricerca non informate

Il problema specifica il grafo e l'obiettivo mentre la **strategia di ricerca** determina il percorso da selezionare dalla frontiera. Le *strategie* **non informate** non prendono in considerazione la posizione dell'obiettivo. Se si considera un costo unitario per ogni arco, possibili strategia sono al ricerca *in ampiezz*, quella in *profondità* o l'*approfondimento iterativo*. Se è disponibileuna funzione di costo, allora si unano strategie dei *costi minimi*.

2.5.1. Ricerca in ampiezza

Nella **ricerca in ampiezza**, detta *breadth-first search, BFS* la frontiera è implementata cn una *coda*, ossia una struttura FIFO. Si seleziona il *primo* percorso aggiunto. I percorsi sono generati nell'ordine del numero di archi contenuti. A ogni passo, si seleziona uno dei percorsi più corti.

Esempio 7. Dal grafo in figura Figura 6, ignorando i costi, si ottiene il grafo nella seguente figura:

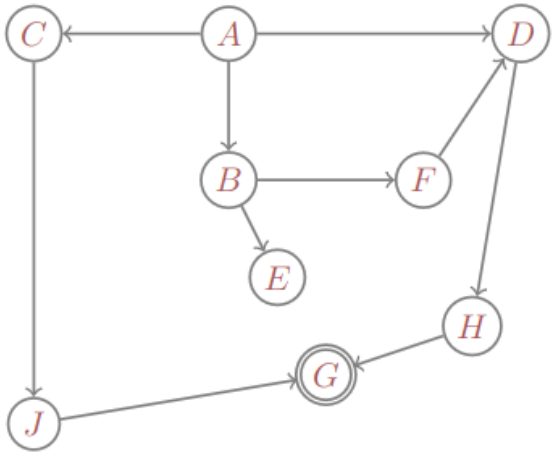


Figura 7: Grafo della figura Figura 6 ma senza informazioni sul costo

In questa figura A è il nodo di partenza e G è il nodo obiettivo. Con la BFS:

- frontiera iniziale: $[\langle A\rangle]$;
- estendendo $\langle A\rangle$ con i suoi vicini si ottiene:

$$[\langle A, B\rangle, \langle A, C\rangle, \langle A, D\rangle]$$

nodi a un arco di distanza da A;

- espandendo questi perocrsi, nell'ordine si ha:

$$[\langle A, B, E\rangle\langle A, B, F\rangle, \langle A, C, J\rangle, \langle A, D, H\rangle]$$

percorsi da A di lunghezza 2;

- dopo l'espansiuone dei percorsi del passo precedente la frontiera diventa:

$$[\langle A, B, F, D\rangle, \langle A, C, J, G\rangle, \langle A, D, H, G\rangle]$$

- selezionando, ad esempio, $\langle A, C, J, G\rangle$ questo percorso viene restituito come soluzione.

La BFS risulta utile quando:
- non si hanno problemi di spazio;
- si cerca una soluzione con numero di archi *minimale*.

La BFS è *poco utile* quando:
- tutte le soluzioni sono associate a percorsi lunghi;
- è disponibile conoscenza euristica;
- il grafo viene generato dinamicamente.

2.5.2. Ricerca in profondità

Nella **ricerca in profondità**, detta *depth-first search*, DFS, la frontiera è implementata con uno *pila*, struttura LIFO: gli elementi vengono aggiunti uno alla volta e quello selezionato e prelevato sarà l'ultimo aggiunto.
Partendo dalla radice, i nodi sono considerati come ordinati da sinistra a destra: il vicino più a sinistra sarà aggiunto in cima per *ultimo*. L'ordine di espansione non dipende dalla posizione dei nodi-obiettivo.
Con una pila, la ricerca procede in profondità:
- *completamento* di un *singolo percorso* prima di provare alternative;
- questo comporta il **backtracking**: si seleziona una prima alternativa per ogni nodo, *tornando indietro* all'opzione successiva solo dopo aver tentato tutti i completamenti;
- l'*ordine di aggiunta* dei vicini alla frontiera/pila non è specificato: questo impatta sull'efficienza e può essere *statico*, ossia prefissato, ovvero *dinamico*, dipendendo dall'obiettivo;
- per una procedura DFS alternativa si veda la figura (inserire figura slide).

Esempio 8 *(DFS sul grafo in Figura 7)* **.** La frontiera come lista con il primo percorso in *cima*. Si parte da $[\langle A\rangle]$. Le frontiere successive sono:
- $[\langle A, B\rangle, \langle A, C\rangle, \langle A, D\rangle]$
- $[\langle A, B, E\rangle, \langle A, B, F\rangle, \langle A, C\rangle, \langle A, D\rangle]$
- $[\langle A, B, F\rangle, \langle A, C\rangle, \langle A, D\rangle]$
- $[\langle A, B, F, D, H\rangle, \langle A, C\rangle, \langle A, D\rangle]$
- $[\langle A, B, F, D, H, G\rangle, \langle A, C\rangle, \langle A, D\rangle]$

Il percorso in cima viene restituito come *soluzione*.

La DFS è *appropriata* in caso di:

- limitazioni di spazio;
- presenza di molteplici soluzioni, anche se costituite da percorsi lunghi;
- ideale quando *tutti* portano a una soluzione;
- oppure se l'ordine di aggiunta dei vicini può essere variato in modo da trovare una soluzione al *primo tentativo*.

La DFS risulta *inefficiente* quando:

- sono possibili percorsi infiniti in caso di grafo infinito o contenente cicli;
- pur esistendo soluzioni alternative poco profonde, la ricerca si attarda su percorsi più lunghi.

2.5.3. Approfondimento iterativo

L'obiettivo dell'algoritmo **ITERATIVE DEEPENING** della DFS è combinare l'efficienza in spazio di DFS con l'ottimalità di BFS. L'idea è quella di non memorizza ma ricalcolare gli elementi della frontiera di BFS usando DFS che usa meno spazio.
DFS effettua una ricerca limitata fino a una data *profondità* come per le iterazioni di BFS. In caso di fallimento si eliminano i percorsi via via calcolati ripartendo, se necessario, dopo aver aumentato il limite. Si parte cercando fino a profondità 1, poi si costruiscono i cammini fino a lunghezza 2, poi quelli di lunghezza 3, ecc.... Se essiste, una soluzione verrà trovata, esplorando percorsi in tale ordine: uno con il numero minimo di archi sarà individuato per primo.
Il *fallimento* della ricerca in profondità limitata può essere:

- *innaturale*: per raggiungimento del limite di profondità e in tal caso la ricerca riparte, dopo aver incrementato il limite;
- *naturale*: quando l'intero spazio di ricerca è esaurito non esiste soluzione a nessun livello di profondità.

Algoritmo 2 — *Ricerca Iterative Deepening*

```
1:  procedure ID_search(G, s, goal)
2:  Input
3:      G: grafo con insiemi di nodi N e di archi A
4:      s: nodo di partenza
5:      goal: funzione booleana sugli stati
6:  Output
7:      cammino da s a un nodo per il quale goal sia vero
8:      ovvero ⊥ altrimenti
9:  Local
10:     raggiunta_max_profondita: boolean
11:     limite_profondita: integer
```

```
12: procedure Ricerca_a_profondita_limitata(⟨n_0, ..., n_k⟩, b)
13:     Input
14:         ⟨n_0, ..., n_k⟩: percorso
15:         b: integer {b ≥ 0}
16:     Output
17:         percorso fino a nodo-obiettivo di lunghezza k + b
18:     if b > 0 then
19:         for each ⟨n_k, n⟩ ∈ A do
20:             risultato ← Ricerca_a_profondita_limitata(⟨n_0, ..., n_k, n⟩, b − 1)
21:             if risultato ≠ ⊥ then
22:                 return risultato
23:     else if goal(n_k) then
24:         return ⟨n_0, ..., n_k⟩
25:     else if n_k ha vicini then {fallimento innaturale}
26:         raggiunta_max_profondita ← true
27:     return ⊥
28: limite_profondita ← 0
29: repeat
30:     raggiunta_max_profondita ← false
31:     risultato ← Ricerca_a_profondita_limitata(⟨s⟩, limite_profondita)
32:     if risultato ≠ ⊥ then
33:         return risultato
34:     limite_profondita ← limite_profondita + 1
35: until not raggiunta_max_profondita
36: return ⊥ {fallimento naturale}
```

La procedura Ricerca_a_profondita_limitata implementa una DFS a profondità limitata: trova percorsi di lunghezza $k + b$, dove k è la lunghezza del percorso e $b \geq 0$; viene chiamato a profondità crescenti. I percorsi vengono trovati nello stesso ordine della BFS: si controlla di aver trovato un obiettivo solo se $b = 0$.
Per assicurare di fallire quando anche la BFS fallirebbe, si tiene traccia dei casi in cui un limite amggiore potrebbe aiutare a trovare una soluzione. La ricerca *fallisce inaturalmente* se ha esaurito tutto lo spazio di ricerca e non sono stati tagliati percorsi per limite raggiungo; se raggiunta_max_profondita, falsa al momento della chiamata, diventa vera alla fine, il limite può essere incrementato per la successiva iterata.

2.5.4. Ricerca di soluzioni a costo minimo

In caso di archi con *costi* non unitari si cerca la soluzione di *minimo costo totale*. Gli algoritmi precedenti non garantiscono soluzioni di costo minimo: il costo non viene preso in considerazione. BFS minimizza solo il *numero* degli archi: potrebbe

esistere una soluzione alternativa, come un percorso più lungo ma di costo inferiore. L'algoritmo **LOWEST COST-FIRST SEARCH** (LcFS), detto anche *least-cost search* o *uniform-cost search*, si comporta come BFS ma con la selezione dei percorsi di costo minimo. Viene implementato usando una *coda con priorità* come frontiera ordinata da *cost*.

> **Esempio 9.** Si consideri il grafo in figura Figura 6. Si espanderà sempre il percorso più a sinistra nella frontiera (costi dei percorsi indicati come pedici):
>
> - $[\langle A\rangle_0]$ frontiera iniziale
> - $[\langle A,B\rangle_2, \langle A,C\rangle_3, \langle A,D\rangle_4]$
> - $[\langle A,C\rangle_3, \langle A,B,E\rangle_4, \langle A,D\rangle_4, \langle A,B,F\rangle_5]$
> - $[\langle A,B,E\rangle_4, \langle A,D\rangle_4, \langle A,B,F\rangle_5, \langle A,C,J\rangle_{10}]$
> - $[\langle A,D\rangle_4, \langle A,B,F\rangle_5, \langle A,C,J\rangle_{10}]$
> - $[\langle A,B,F\rangle_5, \langle A,D,H\rangle_8, \langle A,C,J\rangle_{10}]$
> - $[\langle A,B,F,D\rangle_7, \langle A,D,H\rangle_8, \langle A,C,J\rangle_{10}]$
> - $[\langle A,D,H\rangle_8, \langle A,C,J\rangle_{10}, \langle A,B,F,D,H\rangle_{11}]$
> - $[\langle A,C,J\rangle_{10}, \langle A,D,H,G\rangle_{11}, \langle A,B,F,D,H\rangle_{11}]$.
>
> Dopo l'espansione di $\langle A,C,J\rangle$, si selezionerà $\langle A,D,H,G\rangle$, percorso di costo minimale da A a G.

Il fattore di ramificazione è *finito* se i *costi* sono *limitati* inferiormente da una costante positiva: ciò garantisce la soluzione ottimale, quando esiste. Il primo percorso trovato termina in un nodo-obiettivo ed è ottimale perchè si procede in ordine di costo.
In mancanza di un limite inferiore allora sono possibili percorsi *infiniti*, ad esempio con nodi $n_0, n_1, n_2, ...$ con $\forall i > 0 : \langle n_{i-1}, n_i$ di costo $\frac{1}{2^i}$ esistono infiniti percorsi $\langle n_0, n_1, ..., n_k\rangle$, con costo < 1; se $\exists\langle n_0, g\rangle, goal(g)$ con costo ≥ 1, l'arco non verrebbe mai selezionato.
La complessità è esponenziale in spazio e tempo: genera tutti i percorsi con costo inferiore a quello della soluzione.

2.6. Ricerca Informata (Euristica)

2.6.1. Euristica

Tale tipo di ricerca prende in considerazione *informazioni sull'obiettivo*, nella selezione dei nodi da esplorare, attraverso l'uso di una **funzione euristica** h che associa a ogni nodo n un numero reale *non-negativo*, una stima del costo minimale di un perocorso da n fino a un nodo-obiettivo.
L'euristica h si dice **ammissibile** se *sottostima* il costo reale: $h(n)$ minore o uguale rispetto al costo minimale effettivo del percorso da ciascun nodo a un nodo-obiettivo.

Tale informazione approssimata è spesso immediatamente disponibile trovando un *compromesso* tra efficienza del suo calcolo e accuratezza della stima. Tipicamente, risolvendo una forma semplificata del problema, si possono poi usare i costi effettivi del problema semplificato come euristica nella soluzione del problema originario.

Esempio 10 *(Problema nel grafo in Figura 6)* **.** Considerando come *euristica* la distanza in linea retta tra nodo e obiettivo più vicino, quindi supponendo che tali distanze siano ad esempio:

	A	B	C	D	E	F	G	H	J
$h(\cdot)$	7	5	9	6	3	5	0	3	4

h risulta ammissibile e in particolare: essa è

- *esatta* per il nodo H;
- *molto sottostimata* per il nodo B: sembra vicino al nodo-obiettivo (costo 5), ma raggiungere l'obiettivo costa molto di più;
- *ingannevole* per il nodo E: sembra vicino al nodo-obiettivo (costo 3) ma non permette di raggiungere l'obiettivo.

2.6.2. DFS Euristica e Greedy best-First Search

L'euristica h può essere *estesa* al caso dei percorsi:

$$h(\langle n_0, ..., n_k \rangle) = h(n_k)$$

La **DFS Euristica** usa h per ordinare i *vicini* aggiunti alla pila/frontiera della DFS standard. I vicini vengono aggiunti in modo che il *migliore* vada in cima. Tale scelta è comunque *locale*: si esplorano i percorsi che estendono quello selezionato prima di tentarne altri. Si presentano problemi analoghi al caso della DFS: non c'è garanzia di terminazione e ritrovamento di una soluzione ottimale.
Si potrebbe considerare la *best-first search* che sfrutta un'euristica h da minimizzare nella scelta del prossimo cammino da espandere. In particolare, nella *greedy best-first search*, GbFS, si sceglie il percorso della frontiera con valore di h minimo. Purtroppo potrebbe seguire cammini apparantemente promettenti che, però, potrebbero continuare a estendersi indefinitamente.

2.6.3. Ricerca A^*

L'algoritmo A^* combina idee da LcFS e GbFS. Nella selezione del percorso da espandere si considerano: il costo effettivo del percorso parziale che dal nodo iniziale arriva al nodo corrente e l'euristica, ossia una stima del percorso dal nodo corrente fino a un obiettivo.
Per ogni percorso $p = \langle s, ..., n \rangle$ della frontiera, si stima il costo di un percorso che passi per p e progesua fino a un nodo-obiettivo g:

$$f(p) = cost(p) + h(p)$$

dove $cost(p)$ rappresenta il costo effettivo di p e $h(p)$ è una stima del costo del cammino da n a g.
Si segue lo schema dell'algoritmo di ricerca generico in Algoritmo di ricerca generico e si rappresenta la *frontiera* con una coda con priorità, ordinata da $f(p)$. Se h è ammissibile allora $f(p)$ non sovrastima il cost d'un percorso completo che includa p. A^* può seguire molti perocrsi ma alla fine ne trova uno di costo minimo. Altri possono semprare (provvisoriamente) di costo inferiore. A^* migliora le prestazioni degli algoritmi da cui origina.

Esempio 11 *(A^* su grafo ed euristica)* **.** Si consideri il grafo in figura:

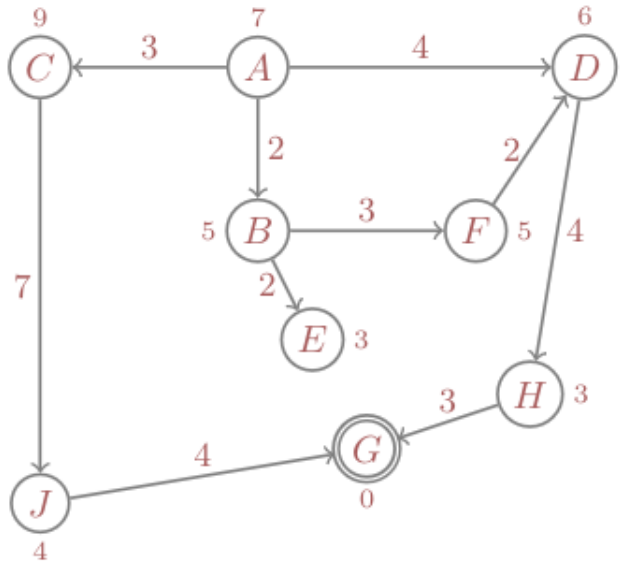


Figura 8: Problema con costi effettivi sugli archi e valori dell'euristica sui nodi

Sequenza delle frontiere. Il pedice indica il valore di $f(p)$.

- $[\langle A\rangle_7]$ iniziale, essendo $h(A) = 7$ e $cost(\langle\rangle) = 0$
- $[\langle A, B\rangle_7, \langle A, D\rangle_{10}, \langle A, C\rangle_{12}]$
- $[\langle A, B, E\rangle_7, \langle A, B, F\rangle_{10}, \langle A, D\rangle_{10}, \langle A, C\rangle_{12}]$
- $[\langle A, B, F\rangle_{10}, \langle A, D\rangle_{10}, \langle A, C\rangle_{12}]$
- $[\langle A, D\rangle_{10}, \langle A, C\rangle_{12}, \langle A, B, F, D\rangle_{13}]$
- $[\langle A, D, H\rangle_{11}, \langle A, C\rangle_{12}, \langle A, B, F, D\rangle_{13}]$
- $[\langle A, D, H, G\rangle_{11}, \langle A, C\rangle_{12}, \langle A, B, F, D\rangle_{13}]$

La soluzione è $\langle A, D, H, G\rangle$ di costo 11.

Un algoritmo è **ammissibile** se, quando esistono soluzioni, ne trova sempre una ottimale.

Proposizione 1. *A^* è ammissibile se:*

- *il fattore di ramificazione è limitato;*
- *i costi degli archi sono maggiori di un certo $\varepsilon > 0$*
- *h è ammissibile, non supera il costo minimale di percorsi da ciascun nodo a un obiettivo.*

A^* garantisce che la prima soluzione sarà ottimale, anche in presenza di cicli*, ma non assicura che ciascun nodo intermedio selezionato dalla frontiera sia un cammino ottimo.*

Importanza dell'euristica

Il miglioramento dell'efficienza in A^* è dato dall'uso di h. Sia c il costo del percorso di costo minimo. Se h è ammissibile, A^* espande ogni percorso dal nodo di partenza nell'insieme

$$\{p \mid cost(p) + h(p) < c\}$$

più alcuni percorsi nell'insieme

$$\{p \mid cost(p) + h(p) = c\}$$

Per l'efficienza di A^*, h è tanto migliore quanto più ridce la cardinalità del primo insieme.

2.6.4. Algoritmo Branch and Bound

L'idea di questo algoritmo è combinare l'effic ienza in spazio delle strategie in profondità con informazione euristica. Risulta efficace quando esistono molti cammini verso un nodo-obiettivo. Questo algoritmo assume che $h(n)$ sia ammissibile.
Questo algoritmo memorizza il percorso di costo minimo verso un nodo-obiettivo trovato e il suo costo che diventa un *limite*: un percorso $p : cost(p) + h(p) \geq limite$ aggiorando *limite*; si continua poi a cercare un'eventuale soluzione migliore.
Si genera così una sequenza di soluzioni via via migliori restituendo quella finale: l'algoritmo viene tipicamente implementato usando la DFS per la ricerca in profondità limitata dal valore di *limite*.

Algoritmo 3 — *Ricerca Depth-First Branch and Bound*

```
1: procedure DFBranchAndBound(G, s, goal, h, limite_0)
2: Input
3:     G: grafo con nodi N e archi A
4:     s: nodo di partenza
5:     goal: funzione di test dei nodi
```

```
    h: euristica sui nodi
    limite_0: limite iniziale (∞ se non specificato)
Output
    percorso di costo minimo da s a un nodo obiettivo se esiste una
    soluzione di costo minore di limite_0, oppure ⊥
Local
    percorso_migliore: percorso o ⊥
    limite: numero reale non negativo
procedure cbsearch(⟨n_0, ..., n_k⟩)
    if cost(⟨n_0, ..., n_k⟩) + h(n_k) < limite then
        if goal(n_k) then
            percorso_migliore ← ⟨n_0, ..., n_k⟩
            limite ← cost(⟨n_0, ..., n_k⟩)
        else
            for each ⟨n_k, n⟩ ∈ A do
                cbsearch(⟨n_0, ..., n_k, n⟩)
{main}
percorso_migliore ← ⊥
limite ← limite_0
cbsearch(⟨s⟩)
return percorso_migliore
```

La procedura *cbsearch*, *cost-bounded search*, comunica attraverso variabili globali; inizialmente, *limite* è impostata a limite_0, stima per eccesso del costo d'una soluzione ottimale.
DFBranchAndBound restituirà, se esiste, una soluzione ottimale con costo inferiore a limite_0. Se limite_0 supera di poco il costo minimo, l'algoritmo non espanderà più archi di A^*. La variabile limite_0 serve a far eliminare percorsi di costo maggiore: trovato un percorso completo, si esplorano solo percorsi con valore di f inferiore a quello del percorso trovato, ossia esattamente i percorsi esplorati da A^* quando trova una soluzione. Quando restituisce $\bot$, se $\text{limite}_0 = \infty$ allora non ci sono soluzioni, se invece limite_0 è finito allora non ci sono soluzioni di costo inferiore a limite_0.
Combinato con ID l'algoritmo può incrementare via via il limite fino a trovare una soluzione ovvero può mostrare che non c'è soluzione.

Esempio 12 *(Robot-consegne)* **.** In un problema più complesso, si considerino:

- *stati* comprendenti le consegne da fare;
- il *costo* dato dalla distanza totale per tutte le consegne;
- un'*euristica* possibile pari al massimo tra:
  1. la distanza massima per consegne ancora da fare non caricate: per ognina, la distanza dalla sua posizione e, da qui, fino alla destinazione;

2. la distanza della destinazione più lontana per le consegne trasportate.

Tale massimo non sovrastima il costo: è la soluzione per un problema semplificato che non considera muri e tutte le altre consegne da fare tranne la più difficile; è appropriato: vanno consegnate quelle trasportate, passate aprendere quelle ancora non caricate e portate a destinazione.
Se il robot può trasportare una sola consegna, un'euristica dovrebbe sommare le distanze per il trasporto di ognuna più la distanza dalla più vicina: non è detto che sarà effettuata per prima.

Esempio 13 *(Navigatore)* **.** Per la minimizzazione del tempo:

- possibile euristica: distanza in linea retta dalla locazione corrente al nodo-obiettivo divisa per la velocità massima, assumendo che ci si possa dirigere a destinazione alla massima velocità;
- euristica più sofisticata: date velocità massime diverse per autostrade e strade locali, si sceglie il massimo tra:
  1. minimo tempo stimato per andare a destinazione su strade locali lente;
  2. minimo tempo utile a raggiungere un'autostrada da una strada locale, per poi raggiungere un luogo vicino alla destinazione e arrivarci da strada locale.

2.6.5. Progettazione della funzione Euristica

Un'**euristica ammissibile** è una funzione non negativa h definita sui nodi, tale che dato un nodo n, $h(n)$ non è mai superiore al costo reale del percorso ottimale da n a un nodo-obiettivo.
Una *procedura standard* per definire un'euristica può essere descritta come segue:

- Si trovano soluzioni di un problema più semplice, ossia con *meno vincoli*, quindi spesso molto più facile da risolvere; una sua soluzione ottimale avrà un costo pari o inferiore a una soluzione del problema originario.
- Si considerino problemi di ricerca su mappe semplificate rappresentati da grafi, in cui si sia *vincolati* a muoversi lungo i dati archi: supponendo che il loro costo sia proporzionale alla lunghezza dei collegamenti stradali, un'euristica ammissibile è costituita dalla *distanza euclidea* tra punti/nodi.

Inserire esempi slide

Un problema in forma semplificata può essere risolto anche attraverso una ricerca più semplice rispetto al problema originario: ricerca ripetuta anche più volte e per tutti i nodi.
Conviene memorizzare (in una memoria cache) tali risultati in un **DB di pattern** che associ ai nodi del problema semplificato il valore dell'euristica.
Nel problema semplificato, spesso si hanno meno nodi: più nodi originari sono mappati su uno stesso nodo di quello semplificato come fosse una *classe d'equivalenza.*

2.7. Potatura dello Spazio di Ricerca

Si possono migliorare gli algoritmi di ricerca considerando i diversi percorsi che possono attraversare un nodo. Non tutti sono utili a risolvere il problema per cui gli altri possono essere subito eliminati.
Essenzialmente vi sono due tipi di **pruning**, ovvero *potatura*, con relative strategie:

- *potare i cicli* per trovare percorsi dal costo minimo;
- *potare cammini molteplici*: per ogni nodo si considererà un solo percorso che lo attraversi, eliminando tutti gli altri.

2.7.1. Potatura dei Cicli

Alcuni dei metodi di ricerca visti in precedenza potrebbero rimanere intrappolati in cicli senza possibilità di uscita anche nel caso di grafi finiti.
Per avere la garanzia di trovare una soluzione in caso di *grafo finito* non andranno presi in considerazione vicini già presenti nel percorso corrente.
Nella **potatura dei cicli**, detta *cycle / loop pruning, CP*, si prevede un controllo addizionale preventivo: dato un nodo da aggiungere, va testata l'occorrenza nel percorso.
Nello shema di **Algoritmo di ricerca generico** si aggiungono alla frontiera solo percorsi $\langle n_0, ..., n_k, n\rangle$, tali che $n \notin \{n_0, ..., n_k\}$, scartando il percorso selezionato; in alternativa si può effettuare il controllo dopo la selezione del nodo.
La compelssità dei controlli aggiuntivi per la verifica della presenza di un nodo nel percorso corrente dipende dal metodo di ricerca adottato:

- *costante* permetodi (DF) che memorizzano un *unico percorso*; si può usare una funzione *hash* oppure si può associare un *bit* a ogni ndoo, che sarà acceso quando viene aggiunto a un percorso e viene spento in caso di backtracking; per cui basterà non espandere nodi con il bit acceso; tale metodo funziona perchè si lavora su un solo percorso;
- *lineare* nella lunghezza del percorso corretne per metodi che gestiscono più percorsi (esponenziali in spazio): serve una ricerca per evitare di aggiungere al percorso parziale un nodo già presente.

2.7.2. Potatura di Percorsi Multipli

Spesso più di un percorso porta a uno stesso nodo. Occorre quindi *eliminare* dalla fronteira ogni percorso che porti a un nodo per il quale ne esista già un altro (di costo inferiore).
Una strategia di **multiple-path pruning** (MPP) può essere implementata gestendo una lista di nodi terminali di percorsi già esplorati, detta **closed list** o *explored set*: inzialmente essa è buota poi, selezionato un percorso $\langle n_0, ..., n_k\rangle$ della frontiera, se n_k è già nella lista esso può essere scartato, altrimenti si aggiunge n_k alla lista e si prosegue.

Algoritmo 4 — *Ricerca con Multiple-Path Pruning*

```
1:  procedure SearchMPP(G, s, goal)
2:  Input
3:      G: grafo con nodi N e archi A
4:      s: nodo di partenza
5:      goal: funzione booleana sui nodi
6:  Output
7:      cammino da s a un nodo per cui goal è vero
8:      oppure ⊥ se non ci sono cammini soluzione
9:  Local
10:     frontier: insieme di cammini
11:     explored: insieme di nodi esplorati
12: frontier ← {⟨s⟩}
13: explored ← ∅
14: while frontier ≠ ∅ do
15:     selezionare e rimuovere ⟨n_0, ..., n_k⟩ da frontier
16:     if n_k ∉ explored then
17:         explored ← explored ∪ {n_k}
18:         if goal(n_k) then
19:             return ⟨n_0, ..., n_k⟩
20:         frontier ← frontier ∪ {⟨n_0, ..., n_k, n⟩ : ⟨n_k, n⟩ ∈ A}
21: return ⊥
```

Si noti che questa strategia non garantisce che non eliminato un percorso di costo minimo. Per garantire di preservare le soluzioni ottimali vi sono alcune alternative:

- assicurare che il primo percorso trovato per un dato nodo si ottimale quindi gli altri si possono eliminare;
- se nella frontiera si dovesse inserire un percorso $p = \langle s, ..., n, ..., m \rangle$, ma fosse già presente $p' = \langle s, ..., n \rangle$ meno costoso della parte fino a n in p, si può eliminare p oppure sostituire in p tale parte con p'.

In LcFS, il primo percorso verso un dato nodo è quello di costo minimo. Pertanto potare altri percorsi non elimina un cammino a costo minimo per il nodo. Ciò assicura di ptoer trovare una soluzione ottimale.
A^* non garantisce che quando si seleziona un percorso verso un nodo per la prima volta, questo sia quello di costo minimo: il teorema sull'ammissibilità lo garantisce solo per i percorsi verso nodi-obiettivo; per quelli verso altri nodi dipende dalle proprietà dell'euristica.
Un'**euristica consistente** h (non negativa) soddisfa il vincolo:

$$h(n) \leq cost(n, n') + h(n')$$

per ogni coppia di nodi n e n'.
Se $h(g) = 0$ per ogni nodo-obiettivo g, allora h sarà consistente e non sovrastimerà mai il costo dei percorsi da un nodo verso un obiettivo.
La consistenza è garantita se h soddisfa la **restrizione di monotonicità**:

$$h(n) \leq cost(n, n') + h(n')$$

per ogni arco $\langle n, n' \rangle$. Ciò è più facile da testare: dipende solo dagli archi, non dalle coppie di nodi considerate.

strategia	**selezione dalla frontiera**	**garanzia soluzione**	**complessità (spazio)**
BFS	primo nodo aggiunto	con meno archi	$O(b^d)$
DFS	ultimo nodo aggiunto	no	$O(bd)$
ID	n/a	con meno archi	$O(bd)$
G_{BFS}	$h(p)$ minimale	no	$O(b^d)$
LcFS	$cost(p)$ minimale	costo minimo	$O(b^d)$
A^*	$cost(p) + h(p)$ minimale	costo minimo	$O(b^d)$
DF B&B	n/a	costo minimo	$O(bd)$
IDA*	n/a	costo minimo	$O(bd)$

Tabella 1: Consuntivo dei diversi algoritmi: si considera la complessità in spazio, con d profondità e b limite superiore per il *fattore di ramificazione*.

> **Proposizione 2.** *Data un'euristica consistente, MPP non impedisce ad A^* di trovare una soluzione ottimale.*

Quindi, nelle condizioni della proposizione sull'ammissibilità di A^* che garantiscono solo il ritrovamento di una soluzione ottimale, se l'euristica è consistente, anche A^* con MPP la potrà trovare.
Nella pratica, A^* include MPP per default.

Confronto fra MPP e CP

- MPP è più generale di CP: un ciclo può essere considerato come un altro percorso verso un dato nodo, destinato a essere potato;
- MPP può essere implementato in modo da poter richiedere un tempo costante;
- MPP è preferibile con metodi in ampiezza: virtualmente tutti i nodi considerati vanno memorizzati;
- CP è preferibile con le strategie in profondità, ad esempio DFS con Branch-and-Bound; con MPP+DFS c'è il rischio di crescita esponenziale per la lista chiusa.

2.7.3. Consuntiva sulle Strategie di Ricerca

Un consuntivo schematico sulle diverse strategie di ricerca e la relativa complessità in spazio è disponibile nella (linkare la tabella inserita prima).

Un'**algoritmo** di ricerca **completo** garantisce il ritrovamento di una soluzione quando esiste. Le strategie che trovano percorsi con numero archi o costo minimale sono complete: nel caso pessimo serve un tempo esponenziale nel numero di archi dei percorsi esplorati. L'esistenza di algoritmi completi con minore complessità dipende dalla risposta alla domanda

$$P \neq NP?$$

2.8. Strategia di Ricerca più sofisticate

Sono possibili alcune estensioni delle strategie di ricerca presentate:

- dato che decidere la direzione della ricerca impatta sull'efficienza, si può pensare a una ricerca retrograda per soluzioni ottimali da qualunque nodo;
- per determinare una buona euristica si può usare la *programmazione dinamica*: metodi basati sulla *decomposizione* del problema in un numero di problemi equivalenti ma di minore complessità.

2.8.1. Direzione della ricerca

Nella **ricerca in avanti**, *forward search*, si comincia da un nodo di partenza e si prosegue fino a raggiungere i nodi-obiettivo. Nella **ricerca retrograda**, *backward search*, si parte da un nodo-obiettivo e si usa il *grafo inverso* alla ricerca del non di partenza; i vicini del nodo-obiettivo considerati per inizializzarela fronteira sono tutti gli altri nodi-obiettivo: $\{n : goal(n)\}$.

La direzione di ricerca è indifferente quando:

1. il numero di nodi-obiettivi è finito;
2. per ogni nodo n si possono generare i *vicini* del **grafo inverso** $\{n' : \langle n', n\rangle \in A\}$.

Le dimensioni dello spazio di ricerca sono date da b^d, dove b è il fattore di ramificazione e d la lunghezza del cammino. Riducendo questi parametri si abbassa la complessità aumentando l'*efficienza*. È possibile che vi sia una differenza tra i fattori da ramificazione uscente ed entrante; in tal caso un principio generale per la scelta della direzione della ricerca è quello di considerare la direzione per cui b risulta inferiore.

2.8.2. Ricerca bidirezionale

Nella **ricerca bidirezionale** si riduce il tempo di ricerca procedendo in entrambe le direzioni: quando le due frontiere si intersecano, si deve ricostruire un in singolo cammino dal nodo di partenza al nodo-obiettivo.

Come si è sicuro che le due frontiere s'incantrano ? Nel caso della DFS è difficile

date le ridotte dimensioni delle frontiere mentre nel caso della BFS è garantito.
Combinando DFS e BFS in direzioni opposte, si garantirebbe l'intersezione, ma la scelta per la direzione del cammino dipende. Per la BFS il fattore di tale scelta è il costo del mantenimento della frontiera (tempo proporzionale a b^k) mentre per la DFS è il costo della ricerca nella frontiera per avere un'intersezione; se si tratta di una ricerca simmetrica bidirezionale si è nell'ordine di $2b^{\frac{k}{2}}$, con risparmi in tempo, ma la complessità rimane esponenziale.

2.8.3. Ricerca in una gerarchia di astrazioni

L'idea è quella di astrarre la definizione del problema, rimuovendo dettagli: una soluzuone parziale del problema potrà essere ottenuta da una soluzione per la versione astratta. Con l'astrazione si punta a risolvere un problema in generale, lascaindo da risolvere problemi più semplici e specifici.
Diverse sono le modalità di astrazione di problemi. Tutte le strategie di ricerca presentate sono utilizzabili ma le specifiche decomposizioni utili non risultano facili da individuare.

Esempio 14 *(Ricerca a isole)* **.** Si consideri una ricerca fra i diversi piani di un edificio. Trovata la soluzione a livello di isola si passa a risolvere ricorsivamente i sotto-problemi in modo analogo:

- informazioni sulle soluzioni trovate a livelli più bassi possono servire a quelle per livelli più alti;
- ai livelli più alti si può usare tale informazione per riformulare un piano di ricerca;
- il processo tipicamente non garantisce soluzioni ottimali perchè considera solo alcune delle decomposizioni possibili.

2.8.4. Uso della programmazione dinamica

La **programmazione dinamica, PD,** fornisce un metodo generale di ottimizzazione basato sulla memorizzazione di soluzioni parziali: una soluzione per un problema più semplice già prodotta dev'essere solo ritrovata e non ricalcolata.
Si può ricorrere alla PD per la costruzione offline di un'**euristica perfetta**, *coast_to_goal()*, che fornisca il costo esatto di un cammino completo di costo minimo su frafi finiti:

$$\text{cost_to_goal}(n) = \begin{cases} 0 & \text{se goal}(n) \\ \min_{\langle n,m\rangle \in A}[cost(\langle n,m\rangle) + \text{cost_to_goal}(m)] & \text{altrimenti} \end{cases}$$

calcolato uasndo LcFS + MPP sul grafo inverso a partire dai nodi-obiettivo: si cerca il cammino di costo minimo da ogni nodo verso i nodi-obiettivo e si converva il valore di *coast_to_goal* per ogni nodo.
Una **policy** è una politica da seguire nella specifica dell'arco da considerare per

ogni nodo; essa è **ottimale** se i suoi costi non sono mai superiori ai costi calcolati usando altre policy.

- *coast_to_goal* viene calcolata offiline e usata nella costruzione di una policy: da n si dovrebbe andare al vicino m che minimizza $cost(\langle n, m\rangle) + \text{cost_to_goal}(m)$. Questa policy porta sempre a un obiettivo con un cammino di costo minimo partendo da qualsiasi nodo;
- sono presenti della alternative:
  1. memorizzare il vicino per tutti i nodi offline, sfruttando le associazioni tra nodi nelle decisioni online sulle azioni, oppure
  2. fornire *cost_to_goal* pre-calcolata e calcolare i vicini online.

Riguardo la complessità si può dire in breve che:

1. PD è lineare in tempo e spazio rispetto alle dimensioni del granfo nella costruzione di *cost_to_goal*, tipicamente esponenziali nella lunghezza del percorso;
2. Data *cost_to_goal*, determinare l'arco migliore richiede tempo costante rispetto alla grandezza del grafo, essendoci un numero limitato di vicini per nodo.

La PD può servire a costruire euristiche per A^* e Branch-and-Bound: semplificando il problema fino a ottenere spazi di ricerca ridotti e trovando in tali spazi soluzioni di lunghezza ottimale; ciò definisce un DB di pattern usato nell'euristica per il problema originale.
La PD è utile quando i nodi-obiettivo possono essere elencati esplicitamente, la soluzione cercata è un cammino di costo minimo, il grafo ridotto è finito, con memoria sufficiente per contenere la tabella, l'obiettivo non cambia. La policy può essere riusata per i diversi nodi-obiettivo: si ammortizza il costo per produrre la tabella su diverse istanze dello stesso problema; tuttavia essa va calcolata per ogni diverso nodo-obiettivo.

Ragionamento con vincoli

3.1. Variabili e vincoli

Vedremo come passare da problemi definiti su spazi di stati a problemi su spazi definiti da caratteristiche. Le *caratteristiche*, o *feature*, vengono descritte attraverso l'uso di **variabili**, spesso non indipendenti fra loro, e di **vincoli rigidi** che specificano combinazioni lecite di assegnazioni alle variabili, e/o **vincoli flessibili**, ossia funzioni che codificano le preferenze fra le diverse assegnazioni.

Il *ragionamento* si svolge generando assegnazioni che soddisfino i vincoli rigidi e ottimizzino i vincoli flessibili.

3.1.1. Variabili e Assegnazioni

Si considereranno problemi descritti in termini di variabili *algebriche* ossia simboli usati per denotare caratteristiche del mondo (reale o immaginario).
La notazione adotta nomi che iniziano per maiuscola: ogni variabile ha un **dominio** associato, denotato con $dom(X)$. Si considereranno *variabili discrete* con dominio finito o almeno enumerabile. Un altro tipo è quello delle *variabili continue*, ad esempio variabili con dominio $\mathbb{R}$.
Un'**assegnazione** è una funzione da un insieme di variabili ai loro domini: dato $\{X_1, X_2, ..., X_k\}$ a X_i si assegna $v_i \in dom(X_i)$ per ogni $i = 1, ..., k$:

$$X_1 = v_1, X_2 = v_2, ..., X_k = v_k$$

Essendo una funzione a ogni variabile viene assegnato un solo valore. Un'**assegnazione totale** riguarda tutte le variabili, altrimenti si dice **parziale**. Un'assegnazione totale rappresenta uno stato del mondo, detto *mondo possibile*.
Date n variabili con domini di cardinalità d, si hanno d^n assegnazioni totali. L'uso delle variabili offre un *vantaggio*: con poche variabili si possono descrivere molti stati:

- con 10 variabili binarie: $2^{10} \approx 10^3$ stati;
- con 20 variabili binarie: $2^{20} \approx 10^6$ stati;
- con 30 variabili binarie: $2^{30} \approx 10^9$ stati;
- con 100 variabili binarie: $2^{100} \approx 10^{30}$ stati.

Ragionare con 30 variabili è più facile che con un miliardo di stati ma anche con 100 variabili non ci sarebbero grossi problemi, mentre è impraticabile ragionare esplicitamente con 2^{100} stati.
Tuttavia molti problemi reali possono essere definiti solo in termini di migliaia o anche milioni di variabili, come le previsioni del meteo.

3.1.2. Vincoli

Dato un problema, le assegnazioni possono essere *ammissibili* o *non ammissibili*.
Un **vincolo rigido** (*hard constraint*) specifica le assegnazioni lecite per una o più variabili e comprende:

- un **ambito**, detto *scope*, ossia l'insieme S di variabili coinvolte con una sua arietà $\mid S \mid$
- la **condizione**, una funzione booleana sulle assegnazioni alle variabili del vincolo che dovrà risultare vera solo per assegnazioni lecite.

Seguono alcuni esempi di vincoli di diverse arietà:

- $B \leq 3$ *unario*;
- $A \leq B$ *binario*;

- $A + B = C$ *ternario.*

La *definizione* di un vincolo può essere: **intensionale**, ossia in termini di formule logiche; **estensionale**, come elencazione delle assegnazioni lecite, come con le relazioni ovvero le tabelle di tuple nei DB relazionali.
Dati il vincolo c con ambito S e l'assegnazione A su variabili contenute in S, anche non tutte, si dirà che A soddisfa c se la condizione è vera per A ristretta all'ambito di c ovvero che A **viola** c in caso contrario.

Esempio 15 *(Gita)* **.** In una gita di quattro giorni, tre *attività* da svolgere possono essere rappresentate da variabili A, B, C tutte con identico dominio $\{1, 2, 3, 4\}$.
Si consideri il seguente vincolo in forma intensionale su $\{A, B, C\}$:

$$(A \leq B) \wedge (B < 3) \wedge (B < C) \wedge \neg(A = B \wedge C \leq 3)$$

esso richede che A non possa seguire B, B sia svolta prima del 3° giorno e comunque prima di C e, infine, che se A è concomitante con B, C si svolga dopo il 3° giorno.
Lo steso vincolo, definito estensionalmente elenca le assegnazioni lecite:

A	B	C
2	2	4
1	1	4
1	2	3
1	2	4

Ad esempio è soddisfatto da $\{A = 1, B = 2, C = 3, D = 3, E = 1$ in quanto in tabella si trova $\{A = 1, B = 2, C = 3\}$, ristretta al suo ambito.

3.1.3. Problema di soddisfacimento di vincoli

Un **problema di soddisfacimento di vincoli (CSP)** è definito da un insieme di variabili ognuna con un proprio *dominio* e un insieme di *vincoli*. Una sua **soluzione** è un'assegnazione totale che soddisfa tutti i vincoli.
Un *CSP finito* ha un numero finito di variabili di dominio finito. Oltre a metodi per CSP finiti, si prenderanno in considerazione anche algoritmi per casi con variabili dal dominio continuo. Molti esempi vengono dal mondo dei giochi, come il Sudoku o la Criptoaritmetica.

Esempio 16 *(Robot consegne)* **.** Si può costruire un CSP definendo:
- le attività da svolgere: a, v, c, d, e e i *momenti* possibili: $1, 2, 3, 4$;
- le rispettive variabili A, B, C, D, E, tutte con lo stesso dominio, ossia con $dom(A) = dom(B) = dom(C) = dom(D) = dom(E) = \{1, 2, 3, 4\}$;

- l'insieme dei *vincoli*:

$\{B \neq 3; C \neq 2; A \neq B; B \neq C; C < D; A = D; E < A; E < B; E < C; E < D; B \neq D\}$

Legati ai CSP, si possono definire diversi problemi di complessità crescente:
- determinare se esista una soluzione o meno;
- trovare una soluzione;
- contare il numero di soluzioni;
- enumerare tutte le soluzioni;
- trovare la soluzione migliore, rispetto a una data misura di qualità;
- determinare se alcuni enunciati siano veri per tutte le soluzioni.

I CSP possono risultare difficili per il loro carattere multidimensionale; il compito base è quello di trovare una soluzione. Già nel caso di CSP con domini finiti tale problema è NP-completo; metodi sistematici hanno una complessità esponenziale ma, ove possibile, si sfrutta la struttura dello spazio di ricerca.

3.2. Risoluzione di CSP tramite ricerca

Un algoritmo **Generate-And-Test** è un algoritmo *esaustivo* per CSP finiti semplce ma inefficiente. Per trovare una soluzione si generano e controllano in modo sistematico, una alla volta, le possibili assegnazioni totali; si restituirà la prima che soddisfa tutti i vincoli. Per trovare tutte le soluzioni si deve continuare a iterare, conservando le soluzioni via via trovate.
Si osservi che con n domini di cardinalità d si hanno d^n possibili assegnazioni totale; con e vincoli il numero totale di test è $O(ed^n)$: al crescere di n diventa rapidamente intrattabile, servono quindi soluzioni alternative.

3.2.1. Algoritmi su Grafo di Ricerca

In alternativa, si possono sfruttare *algoritmi di ricerca su grafo* per spazi di stati determinati dalle assegnazioni parziali. L'idea conduttrice è quella di testare i vincoli su assegnazioni parziali di ambito limitato crescente: se un'assegnazione parziale viola un vincolo, anche le assegnazioni totali che la estendono lo violeranno quindi è possibile potare lo spazio di ricerca.
Si può usare la DFS per cercare tutte le soluzioni di un CSP con variabili V_s e vincoli C_s, estendendo via via l'assegnazione parziale *contesto_parziale*, dove:
- V_s insieme delle variabili senza assegnazione in *contesto_parziale*;
- C_s insieme dei vincoli che coinvolgono almeno una variabile in V_s;
- chiamata iniziale: `DFS_solver` $(V_s, C_s, \emptyset)$.

Rispetto a Generate-And-Test, i test anticipati consentono di potare sotto-alberi, risparmiando lavoro.

Algoritmo 5 — *Risolutore di CSP tramite DFS*

```
1:  procedure DFS_solver(V_s, C_s, contesto_parziale)
2:  ce ← {c ∈ C_s | c  valutabile in  contesto_parziale}
3:  if contesto_parziale viola un vincolo in ce then
4:  |  return ∅
5:  else if V_s = ∅ then
6:  |  return contesto_parziale
7:  else
8:  |  selezionare una variabile var ∈ V_s
9:  |  sols ← ∅
10: |  for val ∈ dom(var) do
11: |  |  sols ← sols ∪
12: |  |       DFS_solver(V_s \ {var}, C_s \ ce, {var = val} ∪ contesto_parziale)
13: |  return sols
```

L'albero di ricerca risultante ha dimensioni che dipendono dall'ordine di scelta delle variabili. Un ordine *statica*, ad esempio sempre prima A, poi B, poi C, risulterà meno efficiente di uno *dinamico*. Ma l'ordine ottimale potrebbe essere più difficile da trovare.

3.3. Algoritmi basati su Consistenza

Si introduce una nozione di **consistenza** intesa come compatibilità, o coerenza logica, fra assegnazioni e vincoli. Una **rete di vincoli** (*constraint network*) indotta da un CSP è costituita da un *grafo bipartito*:

- un nodo *(circolare)* per ogni variabile, con un dizionario *dom* che, per ogni variabile X, contenga l'insieme $dom[X]$ di valori possibili, inizialmente impostato con l'intero suo dominio;
- un nodo *(rettangolare)* per ogni vincolo c;
- un arco $\langle X, c\rangle$ per ogni variabile X nell'ambito del vincolo c.

Un arco $\langle X, c\rangle$ si dice **consistente** *rispetto ai domini (domain consistent)* se e solo se $\forall x \in dom[X] : \{X = x\}$ soddisfa c.
Dato il vincolo c su $\{X, Y_1, ..., Y_k\}$, l'arco $\langle X, c\rangle$ è **consistente** se e solo se $\forall x \in dom[X], \exists y_1, ..., y_k : y_i \in dom[Y_i]$ tale che $\{X = x, Y_1 = y_1, ..., Y_k = y_k$ soddisfi c.
Una **rete consistente** contiene solo archi consistenti. Se $\langle X, c\rangle$ non è consistente allora per qualche valore di $dom[X]$ non ci sono valori di $Y_1, ..., Y_k$ tali che l'assegnazione risultante soddisfi c, quindi *eliminando* tali valori da $dom[X]$ si può ripristinare la consistenza di $\langle X, c\rangle$. È bene notare, però, che l'eliminazione di valori

da un dominio può rendere altri archi non consistenti.
L'idea di base è quella di rendere la rete consistente restringendo i domini. A tale scopo, si considera l'insieme *inconsistenti* degli archi potenzialmente non consistenti:

- si inizializza *inconsistenti* con tutti gli archi del grafo;
- si ripete fino a svuotare *inconsistenti*: estratto un arco $\langle X, c\rangle$ da *inconsistenti* se $\langle X, c\rangle$ non è consistente, si deve restringere $dom[X]$. Si devono, poi, aggiungere a *inconsistenti* gli archi resi non consistenti dal passo precedente: $\langle Z, c'\rangle, c' = c$, con ambito che comprende X e una diversa Z.

Algoritmo 6 — *Generalized Arc Consistency (GAC)*

```
1:  procedure GAC(V_s, dom, C_s, inconsistenti)
2:  while inconsistenti ≠ ∅ do
3:      selezionare e rimuovere ⟨X, c⟩ da inconsistenti
4:      {Y_1, ..., Y_k} ← ambito(c) \ {X}
5:      ND ← {x | x ∈ dom[X] ∧ ∃y_1 ∈ dom[Y_1], ..., y_k ∈ dom[Y_k] :
6:            c(X = x, Y_1 = y_1, ..., Y_k = y_k)}
7:      if ND ≠ dom[X] then
8:          inconsistenti ← inconsistenti ∪ {⟨Z, c'⟩ | {X, Z} ⊆ ambito(c'), c' ≠ c, Z ≠ X}
9:          dom[X] ← ND
10: return dom
```

GAC termina con una rete consistente con variabili dai domini *ridotti*. Tre casi sono possibili:

1. un dominio è vuoto quindi non ci sono soluzioni: se uno è vuoto, lo saranno anche altri domini ridotti ad esso connessi, già prima della terminazione;
2. i domini sono tutti ridotti a un solo valore, quindi la *soluzione è unica*;
3. altrimenti, si è ottenuto CSP semplificato cui applicare altri metodi.

Dei metodi alternativi sono algoritmi basati sulla **consistenza dei percorsi**.

3.4. Separazione dei domini

L'idea base degli algoritmi di **separazione dei domini** è quella di decomporre il CSP in una serie di *casi disgiunti* da risolvere separatamente; le soluzioni sono ricostruire riunendo quelle trovate per i diversi casi. Ad esempio:

- X binaria, dominio $\{t, f\}$, due problemi ridotti: trovare le soluzioni con $X = t$ e quelle con $X = f$; se ne basta una, secondo caso considerato solo se il primo non ha soluzione;
- A con dominio $\{1, 2, 3, 4\}$, ci sono diverse maniere per separare i valori:

- un caso per ciascun valore; ciò fa fare più strada con una sola suddivisione alla volta;
- due sottoinsiemi disgiunti: $A \in \{1, 2\}$ e $A \in \{3, 4\}$; questo taglia di più in meno passi.

Nel seguente schema di algoritmo si integra l'approccio basato sulla consistenza in un algoritmo ricorsivo:

1. si semplifica il CSP in input tramite `GAC()`
2. se non è risolto direttamente:
   1. si seleziona una variabile, con dominio almeno binario;
   2. si partiziona il dominio ottenendo (2+) problemi semplificati;
   3. si risolvono ricorsivamente tali problemi.

ALGORITMO 7 — *Risolutore ricorsivo basato su consistenza*

```
1:  procedure Con_Solve(V_s, dom, C_s, inconsistenti)
2:  dom_0 ← GAC(V_s, dom, C_s, inconsistenti)
3:  if ∃X : dom_0[X] = ∅ then
4:  |  return false
5:  else if ∀X : |dom_0[X]| = 1 then
6:  |  return soluzione con X = x ∈ dom_0[X]   ∀X
7:  else
8:  |  selezionare X tale che |dom_0[X]| > 1
9:  |  partizionare dom_0[X] in D_1 e D_2
10: |  dom_1 ← copia di dom_0 con dom_1[X] = D_1
11: |  dom_2 ← copia di dom_0 con dom_2[X] = D_2
12: |  inconsistenti ← {⟨Z, c'⟩ | {X, Z} ⊆ ambito(c'), Z ≠ X}
13: |  return Con_Solve(V_s, dom_1, C_s, inconsistenti) or
14: |       Con_Solve(V_s, dom_2, C_s, inconsistenti)
```

L'algoritmo fornisce tutte le soluzioni:

- se un dominio è vuoto non ci sono soluzioni;
- se il dominio ha un solo valore si ha una sola soluzione;
- ritornando dalla ricorsione restituisce l'unione delle soluzioni dei 2 casi;
- c'è modo di usare anche algoritmi di ricerca su grafo, ma qui contano le soluzioni e non i cammini.

Un possibile miglioramento è il seguente: se un'assegnazione rende il grafo *non connesso*, ogni componente può essere risolta separatamente. Si ricava facilmente una soluzione ricombinando le soluzioni delle componenti. Il conteggio del numero di soluzioni è fattibile in modo efficiente.

3.5. Eliminazione di Variabili

L'**eliminazione di variabili**, *variabile elimination (VE)*, è una tecnica che semplifica la rete dei vincoli rimuovendo variabili. L'idea di base è quella di eliminare progressivamente le variabili, una alla volta, ottenendo CSP semplificati da risolvere e infine, a partire dalle loro soluzioni, ricostruire quelle dei CSP più complessi: quando si elimina la variabile X si deve costruire un nuobo vincolo sulle rimanenti che rifletta gli effetti dei vincoli su X; esso sostituisce tutti i vincoli su X producendo una rete semplificata; alla fine, ogni soluzione di un CSP ridotto va estesa per ottenere una soluzione del CSP che ricomprenda anche X.
Per eliminare una variabile X:

1. considerate le relazioni relative a tutti i vincoli su X, sia $r_X\left(X, \overline{Y}\right)$ quella ottenuta dal join di tali relazioni, vincolo che codifica l'*influenza* di X su $\overline{Y}$, insieme delle altre variabili nell'ambito di r_X, vicine di X nel grafo dei vincoli;
2. la *proiezione* di r_X su $\overline{Y}$ sostituisce tutte le relazioni in cui occorre X;
3. si ottiene un CSP ridotto, senza la X, da risolvere ricorsivamente. Al ritorno delle chiamate ricorsive si estendono le tabelle-soluzioni per il CSP ridotto tramite join con r_X, per aggiungere la colonna delle assegnazioni a X. Nel caso-base resta una sola variabile, per cui la soluzione da restituire è la tabella con i valori del dominio consistenti con i vincoli.

Esempio 17. Si consideri un CSP con le variabili A, B, C di dominio $\{1, 2, 3, 4\}$ e sia B la variabile da eliminare, inclusa nei vincoli: $A < B$ e $B < C$. Per eliminarla, si fa il join tra le relazioni dei vincoli su B:

A	B
1	2
1	3
1	4
2	3
2	4
3	4

$\bowtie$

B	C
1	2
1	3
1	4
2	3
2	4
3	4

$=$

A	B	C
1	2	3
1	2	4
1	3	4
2	3	4

La proiezione della tabella-join su A e C induce una nuova relazione senza la B:

A	C
1	3
1	4

2	4

Tale vincolo sostituisce gli originari e contiene tutte le informazioni utili al resto della rete; con VE poi si risolve il resto della rete semplificata.
Per avere una o tutte le soluzioni a partire dalla soluzione del CSP ridotto: si memorizza la relazione del join su A, B, C per estendere la soluzione della rete ridotta includendo B.

ALGORITMO 8 — *Eliminazione di Variabili*

```
1:  procedure VE_CSP(V_s, C_s)
2:  Input
3:      V_s: insieme di variabili
4:      C_s: insieme di vincoli su V_s
5:  Output
6:      relazione contenente tutte le assegnazioni consistenti
7:  if |V_s| = 1 then
8:      return join di tutte le relazioni in C_s
9:  else
10:     selezionare X ∈ V_s da eliminare
11:     CX ← {c ∈ C_s | X ∈ ambito(c)}
12:     R ← join di tutti i vincoli in CX
13:     NR ← proiezione di R sulle variabili V_s \ {X}
14:     S ← VE_CSP(V_s \ {X}, (C_s \ CX) ∪ {NR})
15:     return R ⋈ S
```

Seguono alcune osservazioni:

- *caos base*: rimane una sola variabile, quindi una soluzione esiste se ci sono righe nelle relazioni finali e saranno tutte relative a una sola variabile basterà intersecarle;
- caso *ricorsivo*: l'ordine di selezione delle variabili ha un impatto sull'efficienza; al ritorno se bastasse una soluzione, si restituisce solo una tupla di $R \bowtie S$. È garantito che sia parte d'una soluzione; se un valore di R non avesse tuple, non ci sarebbero soluzioni con tale valore.

VE può essere combinato con algoritmi basati su consistenza da usare per semplificare il problema quando si elimina una variabile. Le tabelle intermedie risulteranno più piccole.

3.6. Ricerca Locale

Nel caso di spazi molto grandi, o infiniti, non è pensabile una ricerca sistematica sull'intero spazio. Si può puntare su metodi *mediamente efficienti* per trovare soluzioni, ma *senza garanzie* di ritrovamento anche quando esistono. Perciò tali metodi sono utili quando si sa che, verosimilmente, ce ne sono.
La classe dei metodi di **ricerca locale**, comunemente investigati nell'ambito della *ricerca operativa* e dell'*AI*, comprende molte tecniche, con uno stesso schema-base:

ALGORITMO 9 — *Schema della Ricerca Locale*

```
1:  procedure Local_search(V_s, dom, C_s)
2:  Input
3:      V_s: insieme di variabili
4:      dom: funzione che restituisce il dominio di una variabile
5:      C_s: insieme di vincoli da soddisfare
6:  Output
7:      assegnazione totale che soddisfa i vincoli
8:  Local
9:      A: dizionario di valori indicizzato dalle variabili in V_s
10: repeat {try}
11:     for each X ∈ V_s do
12:         A[X] ← valore (casuale) da dom(X)
13:     {walk}
14:     while not ferma_walk() and A non soddisfa C_s do
15:         selezionare Y ∈ V_s e un valore w ∈ dom(Y)
16:         A[Y] ← w
17:     if A soddisfa C_s then
18:         return A
19: until terminazione
```

- si inizia con un'assegnazione totale di un valore a ciascuna variabile;
- si tenta di migliorare l'assegnazione iterativametne effettuando passi di *miglioramento*, passi *casuali* e *ripartenze* da assegnazioni iniziali differenti.

Ogni iterazione della `repeat` rappresenta un **tentativo** (*try*): con il primo ciclo `for each` si ha l'**inizializzazione casuale** di A; per ogni assegnazione casuale successiva si ha una **ripartenza casuale** in alternativa anche congetture più informate basate su euristiche o conoscenza pregressa, poi migliorate iterando. Nel ciclo `while` si effettua una **ricerca locale** (*walk*) nello spazio delle assegnazioni:

- si seleziona una assegnazione tra i possibili **successori** di A che differiscono per il valore assegnato a una sola variabile;
- si ha lo stop se si è trovata una soluzione o si avvera il criterio di `ferma_walk()`, ad esempio è stato raggiunto un numero massimo di iterate.

La fermata non è garantita: l'algoritmo può divergere se il CSP non ha soluzione. In alcuni casi, anche se ne esistessero, potrebbe rimanere intrappolato in una regione; una garanzia di *completezza* dipende dai criteri di selezione e di stop.
Random Sampling e la versione della ricerca locale in cui:

- `ferma_walk()` risulta sempre vera quindi il ciclo `while` non viene mai eseguito: si continua indefinitamente a provare assegnazioni casuali che possano soddisfare tutti i vincoli;
- l'algoritmo è completo ossia garantisce di trovare la soluzione se questa esiste, tuttavia il tempo richiesto non può essere limitato e tipicamente risulta molto lento;
- la sua efficienza dipende dalle dimensioni dei domini e dal numero di soluzioni esistenti.

Random Walk è la versione della ricerca locale in cui:

- `ferma_walk()` risulta sempre falsa per cui non si hanno ripartenze casuali: si esce dal ciclo `while` solo se si trova una soluzione e, nel ciclo, si ripete la selezione casuale di una variabile e un valore da assegnarle;
- l'algoritmo è completo con passi più veloci rispetto al resampling di tutte le variabili, ma può richiedere più passi, in base alla distribuzione delle soluzioni;
- in alternativa, quando le dimensioni dei domini delle variabili differiscono, si può selezionare a caso una variabile e poi un valore del suo dominio oppure selezionare casualmente una coppia variabile-valore, il che favorisce la selezione di variabili con dominio più grande.

3.6.1. Miglioramento Iterativo

L'approccio del **miglioramento iterativo** è un tipo di ricerca locale che prevede la selezione del *miglior successore* vicino all'assegnazione corrente in termini di una **funzione obiettivo**. Quando la funzione è da minimizzare (una funzione di *costo* o di *perdita*), il metodo prende il nome di *Greedy Descent*, *discesa sbrigativa*. Se, invece, è da massimizzare, il metodo prende il nome di *Greedy Ascent*, o anche **Hill Climbing**. Nel seguito si considereranno solo funzioni da *minimizzare*, per l'altro criterio sarà sufficiente cambiare il segno della funzione. Nei casi di parità di valore della funzione si opera una scelta casuale.
Si distinguono:

- **ottimi locali**, ossia assegnazioni non migliorabili da alcun successore, come i *minimi* o *massimi locali* trovati, rispettivamente, con *greedy descent* o *ascent*;

- **ottimi globali**, quelli con valutazione massima fra tutte le assegnazioni, che risultano comunque anche ottimi locali.

Una funzione di valutazione tipica per i CSP è il numero di **conflitti**, come i vincoli violati: un'assegnazione totale con 0 conflitti è una soluzione. Tale funzione può essere raffinata pesando i vincoli in maniera differenziata. Adottando il *numero di conflitti*, si dirà che un CSP è *soddisfacibile* se si trova un minimo globale con valore nullo ovvero è non soddisfacibile se il minimo globale ha valore positivo: se si trova un minimo locale con valore positivo, non è detto che esso sia globale, e quindi che il CSP non sia soddisfacibile.
Si considera il miglior successore anche quando questo non ha una migliore valutazione rispetto all'assegnazione corrente. È possibile che l'algoritmo trovi ottimi locali che risultano successori reciproci e che continui a passare da uno all'altro senza poter trovare una soluzione. Questo rende l'algoritmo *incompleto*.

3.6.2. Algoritmi stocastici

Per evitare minimi locali che non siano anche globali, negli **algoritmi stocastici** si ricorre a una maggiore casualità. Le mosse casuali previste saranno:

1. il **random restart**, in cui valori scelti a caso per tutte le variabili: costituisce una mossa casuale globale che consente di ripartire da regione anche completamente diverse dello spazio;
2. il **random step**, una mossa casuale locale da alternare a passi di ottimizzazione: utilizzata nel *Greedy descent / ascent* permette pasasi in direzione opposta in modo da sfuggire a minimi / massimi locali.

Integrando *massimo miglioramento iterativo* e mosse *casuali* si definiscono algoritmi per la **ricerca locale stocastica**.

Esempio 18 *(Spazi 2D)* **.** Si veda la seguente figura:

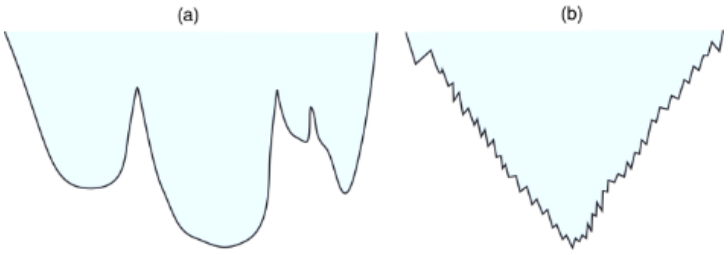


Figura 9: Due spazi di ricerca con caratteristiche diverse

Il *successore* si ottiene tramite un piccolo passo dall'attuale posizione verso sinistra o destra.

- spazio di ricerca (a): *Greedy descent* può trovare facilmente minimi locali, serve un *random restart* che porti nella parte centrale nella quale si converge

rapidamente verso uno globale; invece *random walk* non funzionerebbe bene; richiederebbe molti piccoli passi casuali per uscire da un minimo locale;
- spazio di ricerca (b): *random restart* rimane bloccato a cercare tra numerosi minimi locale mentre *random walk* con *greedy descent* potrebbe evitare tali minimi locali.

Con spazi con diverse caratteristiche in regioni differenti si può pensare di ricorrere a un **catalogo di algoritmi** da cui scegliere.

3.7. Varianti della ricerca Locale

Molte altre varianti sono possibili nella *scelta del successore* e nelle scelte *casuali*. Con domini *limitati* tutti i valori possono essere scelti per i successori. COn domini più estesi si possono considerare solo alcuni valori, risparmiando tempo, in genere si considerano solo quelli più vicini ai precedenti, ma esistono metodi più sofisticati di selezione.

3.7.1. Tabu Search

La **tabu search** è una forma di ricerca locale con memoria che evita la modifica di assegnazioni introdotte di recente. Si memorizzano le variabili modificate negli ultimi t passi (**tenure**) che andranno considerate non selezionabili. In tal modo si evitano i cicli dopo poche assegnazioni. Va ottimizzato il parametro t. Nella sua implementazione, se è piccolo si può considerare una lista delle variabili modificate di recente, invece se è grande si memorizza per ogni variabile il passo in cui si è avuto l'ultima sua modifica.

3.7.2. Passo di Massimo Miglioramento

In questo metodo di ricerca locale, si seleziona una coppia variabile-valore che porta al miglioramento di valutazione massimale. In caso di più coppie si opera una scelta casuale.
Nella sua implementazione «ingenua» data l'assegnazione totale corrente, per ogni variabile X e ogni valore $v \in dom(X)$ diverso da quello corrente, si confronta l'assegnazione corrente con quella in cui si assegna $X = v$. Si seleziona quindi una delle coppie di *massimo miglioramento*. Si noti che la modifica potrebbe portare a *differenze negative* nella valutazione, ossia a peggioramenti. Inoltre, le variabili non coinvolte in vincoli possono essere trascurate.
In un'implementazione alternativa si adotta una coda con priorità fatta di coppie variabile-valore pesate. Per ogni X e ogni $v \in dom(X)$ non assegnato in A, in coda ci sarà una coppia $\langle X, v \rangle$ con peso $w = h(A') - h(A)$. Si considera allora il miglioramento dell'assegnazione A' ottenuta sostituendo $X = v$ rispetto ad A. Questo dipenderà dai valori assegnati a X e dai suoi vicini nella rete dei vincoli, non da

quelli assegnati alle altre variabili. A ogni iterata si seleziona una *coppia-successore* di massimo miglioramento, ossia con *peso minimale*. Ogni nuova assegnazione comporta il ricalcolo dei pesi e il riordimento della coda, ma solo per coppie con variabili presenti in vincoli il cui soddisfacimento è mutato.

3.7.3. Scelta a due fasi

L'algoritmo di **scelta a due fasi** prevede la selezione della coppia articolata in:
1. selezione della variabile per la riassegnazione;
2. selezione del valore.

Si gestisce una coda con priorità di variabili con associato un peso pari al numero dei conflitti in cui ognuna sia coinvolta. Ad ogni passo:
1. si seleziona X che partecipa a più conflitti;
2. le si cambia il valore assegnato, scegiendolo fra quelli che minimizzano il numero di conflitti oppure casualmente.

Vanno infine ricalcolati i pesi per le variabili coinvolte in vincoli il cui soddisfacimento è mutato.

3.7.4. Algoritmo Any-Conflict

L'idea-base dell'algoritmo **Any-Conflict** è quella di scegliere, per la modifica, una **variabile conflittuale** ossia una che partecipa a uno o più conflitti. A ogni iterata: si seleziona casualmente una variabile conflittuale, non necessariamente quella con più conflitti, quindi le si assegna, in alternativa, un valore che minimizzi il numero di conflitti oppure un valore casuale.
Sono possibili alcune varianti, ad esempio cambiando il criterio di selezione casuale della variabile: si può scegliere prima un conflitto e poi una variabile coinvolta oppure si opera una semplice scelta casuale di una variabile conflittuale. La differenza fra i due criteri sta nella probabilità di selezione di una variabile: nel primo essa dipende dal numero di conflitti in cui la variabile è coinvolta, nel secondo si ha la stessa probabilità per tutte le variabili.

3.7.5. Simulated Annealing

Il **simulated annealing** prende in prestito una metafora dal dominio della *metallurgia* (ovvero della *termodinamica*): ad alte temperature l'algoritmo deve comportarsi con maggiore casualità o plasticità mentre a basse temperature deve consentire minore casualità in conseguenza una maggiore durezza. Come funzione di valutazione si adotta un'euristica basata sul numero di conflitti.
L'algoritmo riduce lentamente la temperatura cui è legata una misura di probabilità:
- ad *alte temperature* si deve comportare come *random walk* per poter evitare i minimi locali, alla ricerca di regioni con bassi valori dell'euristica, quindi i passi peggiorativi dovranno risultare più probabili;

- a *basse temperature* si deve comportare come *greedy descent*, portando direttamente verso i minimi (locali).

A ogni passo, data l'assegnazione corrente A: si sceglie a caso una variabile e un valore ottenendo una nuova assegnazione A'; se A' non peggiore l'euristica andrà a rimpiazzare l'assegnazione corrente, altrimenti può farlo ma con una probabilità che dipende dalla temperatura e dal valore dell'euristica.
Si consideri la seguente figura:

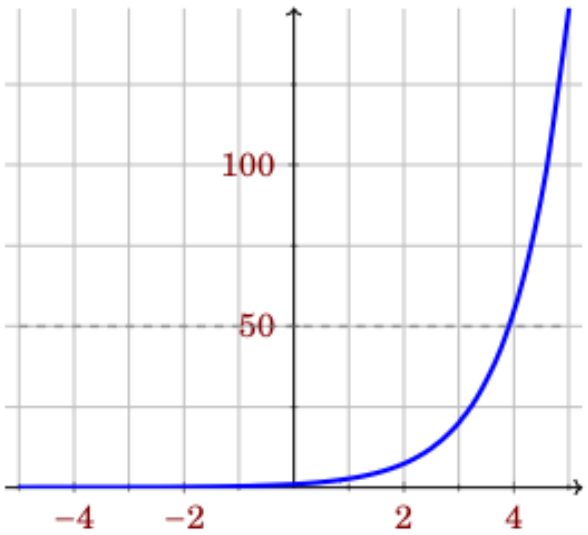


Figura 10: Una funzione esponenziale

Sia la **temperatura** $T \in \mathbb{R}_+$. Si consideri l'andamento della Figura 10.

- Sia $h(A)$ l'euristica da minimizzare, il numero di conflitti legati ad A;
- Se $h(A') \leq h(A)$, si accetta direttamente la nuova A' altrimenti, la si può accettare con probabilità (**distribuzione di Gibss / Boltzmann**):

$$\exp\left[-\frac{h(A') - h(A)}{T}\right]$$

Si noti che quando A' è peggiorativa, quindi l'esponente è negativo. Tendendo $h(A') - h(A)$ a 0, sarà più probabile accettare A'. Infatti ad alte temperature l'argomento dell'esponenziale tende a 0 e la probabilità a 1, mentre a basse temperature, l'esponente tende a $-\infty$ e la probabilità a 0.
Il cosidetto **problema di annealing** specifica cme ridurre la temperatura al progredire della ricerca: spesso si usa il *raffreddamento geometrico* che, ad esempio parte da $T = 10$ e si moltiplica per 0.99 ad ogni passo, arrivando a 0.07 dopo 500 passi. - a temperature alte, come $T = 10$, si tende ad accettare passi che peggiorano di poco, con una leggera preferenza rispetto a passi che migliorano;

- a temperature ridotte, come $T = 1$, i passi peggiorativi sono accettati molto meno frequentamente;
- a temperature basse, come $T = 0.1$, i passi peggiorativi sono accetti molto raramente.

3.7.6. Ripartenza casuale

L'algoritmo di **ripartenza casuale**, *random restart*, permette di migliorare le prestazioni di un *algoritmo causale debole*, uno che abbia successo in pochi casi specifici. Nel seguito sarà indicata con p la probabilità di successo d'una singola esecuzione. Per stimare le prestazioni di *random restart*, si considera una sequensza di n sue esecuzioni indipendenti.
La probabilità di successo in almeno una di tali esecuzioni sarà $1 - (1 - p)^n$ essendo $(1 - p)^n$ la probabilità di fallimento in tutti gli n tentativi.
Random restart risulta computazionalmente costoso quando sono coinvolte molte variabili. Nella sua variante *partial restart* si fanno assegnazioni solo ad alcune variabili, per consentire di spostarsi verso un'altra regione. In tal caso le esecuzioni non sono più indipendenti fra loro quindi si richiede un'analisi teorica più complessa.

3.8. Algoritmi basati su popolazioni

A differenza degli algoritmi esaminati in precedenza che considerano un'assegnazione alla volta, tali metodi gestiscono **popolazioni** di **individui**, ossia insiemi di assegnazioni. Nella *beam search* si considerano le migliori k assegnazioni, numero che può variare casualmente nella loro variante stocastica. Anche negli algoritmi evoluzionistici si considerano i migliori k individui nella metafora riproduttiva.

3.9. Beam Search

La **beam search** si comporta in maniera simile agli algoritmi di massimo miglioramento iterativo, con la differenza di lavorare simultaneamente su più assegnazioni (fino a k) anziché una sola. Essa ha successo quando viene trovata un'assegnazione che soddisfa tutti i vincoli. A ogni passo, si selezionano i migliori k successori. La selezione è casuale in caso di parità di valutazione. Si continua con il nuovo insieme di k assegnazioni.
In caso di *memoria limitata* si sceglie k in base alla memoria disponibile. È possibile usre le diverse varianti di ricerca locale viste in precedenza. Occorrerà ovviamte più tempo nel cercare i migliori k successori. Per una ricerca più rpida, si può ricorrere ad approssimazioni dei migliori k individui.

3.9.1. Beam Search Stocastica

In questa variante si selezionano k individui casualmente favorando quelli con valutazione migliore. La probabilità di selezione sarà definita in funzione dell'euristica: per un individuo A la probabilità di selezione (*distribuzione di Gibbs/Boltzmann*) proporzionale a:

$$\exp\left[\frac{-h(A)}{T}\right]$$

dove $h(A)$ è la funzione obiettivo e T una misura della temperatura. Questa variante consente più diversità nella popolazione: h riflette la capacità di *adattamento* come in biologia dove vale il principio di *sopravvivenza dei più adatabbili* per cui un individuo più adattabile ha maggiori probabilità di passare il proprio materiale genetico alle future generazioni.
Questa è interpretabile come una forma di *riproduzione asessuata* in cui si produce una discendenza leggermente mutata. Si osservi che gli stessi individui potranno essere selezionati casulamente anche più volte.

3.9.2. Algoritmi Genetici

Anche questi metodi sono definiti per gestire *popolazioni* di individui. Restando nella metafora evoluzionistica, il loro patrimonio genetico è rappresentato da un'assegnazione. Nuovi individui vengono creati dalla *combinazione* di *coppie* di *genitori* della generazione precedente. I migliori sono poi selezionati per far parte della generazione successiva.

Algoritmo 10 — *Schema di algoritmo genetico*

```
1:  procedure Genetic_algorithm(V_s, C_s, S, k)
2:  Input
3:      V_s: insieme di variabili
4:      C_s: insieme di vincoli da soddisfare
5:      S: programma di raffreddamento della temperatura
6:      k: dim. popolazione - intero pari
7:  Output
8:      assegnazione totale che soddisfa i vincoli
9:  Local
10:     Pop, NPop: insieme di assegnazioni
11:     T: numero reale
12: Pop ← insieme di k assegnazioni totali casuali
13: T inizializzato in base a S
14: repeat
15:     if ∃A ∈ Pop che soddisfa tutti i vincoli in C_s then
```

```
        return A
    NPop ← ∅
    repeat k/2 volte
        A_1 ← Random_selection(Pop, T)
        A_2 ← Random_selection(Pop, T)
        N_1, N_2 ← Crossover(A_1, A_2)
        NPop ← NPop ∪ {mutate(N_1), mutate(N_2)}
    Pop ← NPop
    T viene aggiornato in base a S
until terminazione
procedure Random_selection(Pop, T)
    selezionare A da Pop con probabilità proporzionale a e^{-h(A)/T}
    return A
procedure Crossover(A_1, A_2)
    selezionare casualmente un intero i, 1 ≤ i < |V_s|
    N_1 ← {(X_j = v_j) ∈ A_1 | j ≤ i} ∪ {(X_j = v_j) ∈ A_2 | j > i}
    N_2 ← {(X_j = v_j) ∈ A_2 | j ≤ i} ∪ {(X_j = v_j) ∈ A_1 | j > i}
    return N_1, N_2
```

Sono possibili diverse tecniche di selezione:

- *Selezione proporzionale alla fitness*: a ogni iterata, si generano k nuovi individui. Si opera una seleezione casuale delle coppie, che favorisca gli individui più adatti con probabilità dipendente dall'incremento della misura di fitness apportato e da una misura della temperatura. Per ogni coppia, si opera un *crossover* e si fanno *mutare* casualmente alcuni valori, per alcune variabili scelte a caso. Si passa infine a considerare la successiva generazione.
- *Selezione a torneo*: a ogni iterata si selezionano casualmente t individui (misura di *greediness*) e poi si scelgono i più adattabili fra questi.

Un'operazione di combinazione tipicamente utilizzata è il **crossover** che prevede la selezione di una coppia di individui e la conseguente generazione della loro prole ossia di individui ottenuti copiando parte delle assegnazioni da un genitore e il resto dell'altro. Si tratta di un'operazione aggiuntiva rispetto alla mutazione. Vi sono diverse forme di crossover:

- **crossover uniforme**: si considerano due individui genitore e si generano due figli; per ogni variabile, si copia nel primo figlio il valore da uno dei genitori scelto in maniera casuale, quello dell'altro genitore sarà copiato nel secondo figlio;
- **one-point crossover**: assumendo un *ordinamento* sulle variabili, si seleziona casualmente un indice i per generare ciascun figlio selezionando i valori per le variabili fino a i da un genitore e per le successive dall'altro. L'efficacia dipenderà dall'ordinamento scelto in fase di progettazione.

3.10. Ottimizzazione

A volte si hanno informazioni sulla preferibilità delle assegnazioni. Un **problema di ottimizzazione** richiede di trovare le migliori assegnazioni totali. Esso può essere formalizzato come segue, dati:

- un insieme di variabili con dominio associato;
- una **funzione-obiettivo** dalle assegnazioni totali a $\mathbb{R}$, tipicamente una funzione di costo/errore/loss;
- un **criterio di ottimalità**, tipicamente *minimizzare* la funzione-obiettivo.

Bisogna trovare un'assegnazione totale **ottimale** per il criterio adottato.
Un **problema di ottimizzazione vincolato** prevede anche *vincoli rigidi* che specificano le assegnazioni ammissibili. L'obiettivo è quello di trovare un'assegnazione ottimale ammissibile.
Sul tema dell'ottimizzazione esiste una vasta letturatura scientifica, con molte tecniche proposte: ad esmepio nella programamzione lineare sono coinvolte variabili continue, funzioni-obiettivo lineari e diseguaglianze lineari come vincoli. Quando un problema da risolvere, prevedendo anche qualche trasformazione, rientra in categorie classiche, allora converrà preferire algoritmi specifici.
Rispetto ai CSP, l'ottimizzazione presenta un'ulteriore difficoltà: riconoscere le soluzioni solo per confronto fra assegnazioni. Si possono trattare i vincoli rigidi come quelli flessibili ma assegnando un costo infinito in caso di violazione: se il costo è finito non vi sarà stata nessuna violazione. In alternativa si può associare alla violazione di vincolo rigido un costo elevato maggiore della somma di tutti i costi di vincoli flessibili. L'algoritmo di ottimizzazione dovrà trovare una soluzione con il minor numero di vincoli rigidi violati e, tra questi,, quelli di costo minimo.

3.10.1. Metodi Sistematici per il Caso Discreto

Si parla di **ottimizzazione discreta** in caso di problema che coinvolge solo variabili *discrete*. Possono essere adattati i metodi di ricerca di percorsi ottimali su grafo visti in precedenza. In questo caso il grafo avrà nodi contenenti assegnazioni a un insieme di variabili. I vicini di un nodo n si trovano scegliendo una variabile *var* non presente in n e le relative assegnazioni di valori, consistenti con gli eventuali vincoli rigidi. Il costo di un arco è dato dalla somma dei costi dei vincoli valutati dopo l'assegnazione a *var*.
Il nodo di partenza conterrà l'assegnazione vuota metnre un nodo-goal conterrà un'assegnazione totale. I costi sono assengati solo quando un vincolo flessibile può essere valutato. Per usare gli algoritmi A^*, *Branch-and-Bound* i costi-arco devono essere non-negativi.

3.10.2. Ricerca locale per l'ottimizzazione

L'uso della ricerca locale nel caso di problemi di ottimizzazione mira a minimizzare la funzione-obiettivo. Gli algoritmi di ricerca lcoale aggiornano via via e infine restituiscono la *migliore assegnazione trovata*. La terminazione può essere garantita fissando un numero massimo di iterate.
È difficile stabilire se un'assegnazione totale trovata sia ottimale: si ha un **ottimo locale** trovando un'assegnazione non peggiore di tutte le successive possibili; un **ottimo globale** è dato da un'assegnazione non peggiore di tutte le assegnazioni. Senza la ricerca sistematica non si può sapere se la migliore assegnazione trovata localmente costituisca un ottimo globale o se ne esiste altrove una migliore.
Per arrivare a una soluzione si può consentire la violazione di vincoli rigidi, ad esempio adottando costi di violazione alti ma finiti.

3.10.3. Domini Continui: Discesa di Gradiente

In questo caso, la ricerca locale è più complicata: bisogna stabilire come definire il *successore* di un'assegnazione.
L'idea dell'algoritmo di *Discesa del Gradiente (Gradient Descent)* è quella di minimizzare la funzione di valutazione (purchè continua e differenziabile). Come in un percorso in discesa, si fanno passi nelle direzioni più ripide. Il successore di un'assegnazione viene calcolato facendo un passo in discesa di lunghezza proporzionale alla *pendenza* della funzione, ossia alla sua derivata.
Esiste anche una ricerca di massimi di funzione obiettivo tramite *Risalita del Gradiente (Gradient Ascent)* facendo passi di segno positivo.
Nel caso *monodimensionale* se a X è assegnato $v \in \mathbb{R}$ il valore successivo sarà:

$$v - \eta \cdot \frac{df}{dX}(v)$$

dove η è la misura del pass che determina la rapidità della discesa; la derivata viene valutata in v:

$$\lim_{\varepsilon \to 0} \frac{f(X = v + \varepsilon) - f(X = v)}{\varepsilon}$$

Nel caso *multidimensionale* si fanno passi in tutte le direzioni, proporzionali a ogni *derivata parziale*. Date le variabili $\langle X_1, ..., X_n \rangle$ e un'assegnazione iniziale di valori $\langle v_1, ..., v_n \rangle$, l'assegnazione successiva si ottiene muovendosi lungo ogni direzione, in proporzione alla relativa pendenza di h. Il nuovo valore per X_i si ottiene calcolando:

$$v_i \leftarrow v_i - \eta \cdot \frac{\partial h}{\partial X_i}(\vec{v}) \quad \forall i = 1, ..., n$$

dove $\frac{\partial h}{\partial X_i}$ è la *derivata parziale* rispetto a X_i (le altre vengono trattate come costanti):

$$\frac{\partial h}{\partial X_i}(\vec{v}) = \lim_{\varepsilon \to 0} \frac{h(v_1, ..., v_i + \varepsilon, ..., v_n) - h(v_1, ..., v_i, ..., v_n)}{\varepsilon}$$

L'utilità dell'algoritmo sarà evidente nell'apprendimento automatico del valore dei parametri di un modello, anche nel caso di modelli complessi con 10^{12} parametri da ottimizzare.
Vi sono numerose **varianti**, ad esempio per η si può usare una forma di ricerca binaria per cercare un valore ottiamle, oppure potrebbe variare in funzione delle diverse iterate.
Per quanto riguarda la convergenza in caso di funzioni regolari con un minimo, GD converge a un minimo locale se η è sufficientemente piccolo. Qunado è troppo grande allora è possibile che diverga, se, invece è troppo piccolo esso converge molto lentamente. Se il minimo locale è unico allora sarà anche un minimo globale. In caso di più minimi locali è necessaria un'ulteriore ricerca per trovare il minimo globale. Ad esempio, si può usare *random restart* o *random walk*. È garantito il minimo globale solo quando sarà stato attraversato l'intero spazio di ricerca.

Rappresentazione e Ragionamentoi Proposizionale

4.1. Proposizioni

Le **proposizioni** sono *enunciati* che possono essere veri o falsi. Esse sono espresse in un linguaggio formale che si può descrivere in termini di sintassi e semantica, la **logica proposizionale**.
Il *ragionamento*, che potrà essere definito in termini di proposizioni, si propone obiettivi quali trovare modelli, trovare conseguenze logiche, formulare ipotesi e fare debugging della conoscenza descritta per correggere eventuali errori.

4.1.1. Sintassi del Calcolo Proposizionale

A ciascun enunciato, se formulato correttamente in un linguaggio basato su variabili booleano e connettivi logici, potrà poi essere associato un valore di verità.
Una **proposizione atomica**, o **atomo** viene rappresntato da un simbolo, condizionalmente una stringa alfanumerica, contenente eventuali «_», che inizi con una lettera minuscola. Esempi di atomi ben formati sono: *ai_divertente*, *piove*, *soleggiato*. Una **proposizione** o **formula logica** è costituita da atomo oppure da una proposizione **composta** formata usando connettivi logici e proposizioni più semplici. Nella seguente tabella sono elencate le diverse proposizioni composte che si possono formare a partire da due generiche proposizioni p e q:

sintassi	lettura	definizione
$\neg p$	«non p»	*negazione di p*
$p \wedge q$	«p e q»	*congiunzione di p e q*
$p \vee q$	«p o q»	*disgiunzione di p e q*
$p \rightarrow q$	«p implica q»	*implicazione di q da p*
$p \leftarrow q$	«p se q»	*implicazione di p da q*
$p \leftrightarrow q$	«p se e solo se q»	*equivalenza di p e q*
$p \oplus q$	«p XOR q»	*or esclusivo di p e q*

Vale il classico ordine di *precedenza* fra connettivi: $\neg, \wedge, \vee, \rightarrow, \leftarrow, \leftrightarrow, \oplus$. Si possono usare le parentesi per disambiguare e avere una maggiore leggibilità, ad esempio la proposizione

$$\neg a \vee b \wedge c \rightarrow d \wedge \neg e \vee f$$

si potrà scrivere nella forma equivalente

$$((\neg a) \vee (b \wedge c)) \rightarrow ((d \wedge (\neg e)) \vee f)$$

4.1.2. Semantica del Calcolo Proposizionale

La **semantica** definisce il significato degli enunciati, fornendo la corrispondenza tra simboli e stato del mondo che viene rappresentato. Ogni proposizione sarà *interpretabile* come vera o falsa rispetto allo stato del mondo: le proposizioni vere determinano il mondo conosciuto dal sistema, essendo un *atomo* l'unità minima che possa essere dotata di significato. Il significato delle proposizioni più complesse discenderà da quello degli atomi.

Un'**interpretazione** è definita come funzione: π : Atomi $\rightarrow$ {true, false}

- se $\pi(\alpha) =$ true, si dice che α **vero** nell'interpretazione π;
- se $\pi(\alpha) =$ false, si dice che α **falso** nell'interpretazione π.

Un'interpretazione si può anche definire come l'insieme I di tutti e soli gli atomi veri, ossia associati al valore di verità true. Tutti gli altri atomi saranno interpretati come associati a false.

Per associare i valori di verità alle proposizioni non atomiche, si dovranno considerare le tavole di verità dei connettivi, mostrate di seguito:

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \leftarrow q$	$p \rightarrow q$	$p \leftrightarrow q$	$p \oplus q$
true	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*true*	*false*	*false*	*true*
false	*true*	*true*	*false*	*true*	*false*	*true*	*false*	*true*
false	*false*	*true*	*false*	*false*	*true*	*true*	*true*	*false*

Si noti che la verità di una proposizione dipende solo dall'interpretazione dei suoi atomi. Una stessa proposizione ammette diverse interpretazioni per gli atomi contenuti, quindi potrà avere assegnati valori di verità diversi.
Una **base di conoscenza** (KB, *knowledge base*) è costituita da un insieme di proposizioni dette **assiomi**: essi sono dichiarati come veri. La base di conoscenza può essere vista come una congiunzione implicita degli assiomi, quindi sarà vera se tutti i suoi assiomi sono veri.
Gli assiomi servono a specificare il mondo da rappresentare attraverso quella che si dice la sua **interpretazione intesa**, ossia quella che dovrebbe risidere nella mente ell'esperto di dominio e che va formalizzata dall'ingegnere della conoscenza fornendo la sua assiomatizzazione. Nella **teoria dei modelli**, un **modello** di una base di conoscenza KB è un'interpretazione per la quale sia vero ogni assioma di KB.
La proposizione g, *goal*, è **conseguenza logica** di KB, ovvero g segue logicamente da KB se e solo se g vera in ogni modello di KB, in simboli:

$$KB \models g$$

In inglese questa relazione fra KB e g si dice *entailment*.
Quindi, se vale $KB \models g$, non esistono modelli di KB nei quali g sia falsa. In caso contrario, si può scrivere $KB \not\models g$ per dire che «g non è conseguenza logica di KB», ciò significa che esiste un modello di KB in cui g è falsa. Un caso da distinguere è quello in cui $KB \models \neg g$.
È bene notare che non vengono prese in considerazione altre possibili interpretazioni di g che non siano modelli di KB.

Esempio 19 *(Base di conoscenza)* **.** Data la KB contenente gli assiomi:

- *sam_felice*;
- *ai_divertente*;
- *vermi_vivono_sottoterra*;
- *sera*;
- *bruco_mangia_mela*;
- *mela_mangiata* $\leftarrow$ *bruco_mangia_mela*;
- *interruttore_acceso* $\leftarrow$ *sam_nella_stanza* $\wedge$ *sera*

si può scrivere:

- $KB \models$*bruco_mangia_mela*
- $KB \models$ *mela_mangiata*

mentre $KB \not\models$*interruttore_acceso* ossia esiste un modello di KB in cui *interruttore_acceso* è falso; tale modello, dovendo risultare vero l'ultimo assioma in KB, può essere definito assumento che *sam_nella_stanza* sia falso.

4.1.3. Semantica: punto di vista del progettista

Per caratterizzare un dominio / mondo, il progettista della *KB* lo definisce come **interpretazione intesa**: il significato dei simboli viene definito attraverso proposizioni, ossia assiomi che si possono assumere veri rispetto a tale stato del mondo, quindi le conseguenze logiche di *KB* saranno vere rispetto all'*interpretazione intesa*. Il significato viene così condiviso e potrà essere usato per interpretare le risposte a interrogazioni della *KB*.

Metodologia di Costruzione di Basi di Conoscenza

Il progetto della base di conoscenza *KB* si sviluppa nelle seguenti fasi:

1. SI sceglie il dominio/mondo da rappresentare, ossia l'interpretazione intesa. Questo può comprendere sia aspetti del mondo reale, un mondo immaginario oppure un dominio astratto come quello numero e/o dell'insiemistica.
2. Si scelgono gli atomi per rappresentare le proposizioni d'interesse, la cosiddetta **concettualizzazione**, cui associare un significato preciso rispetto all'interpretazione intesa.
3. Si definiscono gli assiomi di *KB*, ossia proposizioni vere nell'interpretazione intesa, l'**assiomatizzazione** del dominio applicativo per il sistema.
4. Si pongono al sistema domande sull'interpretazione intesa; le sue risposte dovranno essere interpretate in base ai significati associati ai simboli.

4.1.4. Semantica: Punto di vista della Macchina

Il sistema elabora la *KB* per decidere se g è vera nell'interpretazione intesa. I modelli di *KB* rappresentano tutti e soli i modi in cui il mondo potrebbe essere fatto. Data una concettualizzazione e un'assiomatizzazione corrette, l'interpretazione intesa sarà uno dei modelli di *KB*, ma il sistema non sa determinarlo esattamente.

Il sistema deve determinare se $KB \models g$: se $KB \models g$, esso potrà concludere che g sarà vera nell'interpretazione intesa, essendo g vera in tutti i modelli di *KB*. Se, invece, $KB \not\models g$, allora non può trarre conclusioni: g potrebbe essere falsa proprio nell'interpretazione intesa ma il sistema non può stabilirlo dato che non la conosce. Queste forme di inferenza sono implementate dalle procedure di dimostrazione trattate nel seguito.

È bene notare che $KB \not\models g$ non comporta necessariamente che $KB \models \neg g$.

Inoltre, si noti che il progettista può sbagliare e codificare nella *KB* assiomi falsi nell'interpretazione intesa. In tali casi non può essere garantito che le risposte della macchina siano vere in tale interpretazione.

4.2. Vincoli Proposizionali

Si considereranno ora CSP con vincoli rappresentati come *formule logiche* dalla struttura concisa e formale per consentire l'elaborazione automatica.
Un *problema* di **soddisfacibilità proposizionale (SAT)** è definito da:

- un insieme di *variabili booleane*; si usa la sintassi $X = true$ abbreviata in x, e $X = false$ oppure $\neg x$.
- un insieme di *vincoli clausali* o **clausole**, ossia espressioni logiche della forma:

$$l_1 \vee l_2 \vee \cdots \vee l_k$$

  in cui ogni **letterale** l_i è un atomo a o la sua negazione $\neg a$, ossia un'assegnazione a una variabile booleana; si dirà che a *occorre negativamente* (risp. *positivamente*) nel letterale $\neg a$ (risp. a).

La clausola $l_1 \vee l_2 \vee \cdots \vee l_k$ è vera in un'interpretazione se e solo se $\exists i \in \{1, ..., k\}$: l_i vero in tale interpretazione. In tal casol, si dice anche che il vincolo clausale è *soddisfatto*.
In Logica Proposizionale, una clausola è una forma logica in una forma ristretta. Esiste un algoritmo di conversione per cui ogni proposizione può essere convertita in forma di clausola.
Nei CSP, una clausola è un vincolo su un insieme di variabili booleane che è soddisfatto se almeno uno dei suoi letterali è vero: $l_1 \vee l_2 \vee \cdots \vee l_k$ equivale a $\neg(\neg l_1 \wedge \neg l_2 \wedge \cdots \wedge \neg l_k)$, ciò esclude che le assegnazioni in cui tutti i letterali risultino falsi.

4.2.1. Forma Clausale per CSP

Un CSP finito si può trasformare in un problema di soddisfabilità di una KB proposizionale in cui:

- per ogni variabile Y del CSP, con $dom(Y) = \{v_1, ..., v_k\}$ si considerano le **variabili indicatrici** booleane $\{Y_1, ..., Y_k\}$ tali che $\forall i = 1, ..., k : Y_i$ vera se $Y = v_i$ e falsa altrimenti; quindi per rappresentare Y in KB si considereranno gli atomi $y_1, ..., y_k$;
- si aggiungeranno a KB le seguenti clausole:
  - $\neg y_i \vee \neg y_j \big(\equiv \neg\big(y_i \wedge \neg y_j\big)\big)$ per ogni $i, j \in \{1, ...k\}$ con $i < j$, poichè y_i e y_j non possono essere entrambi veri;
  - $y_1 \vee \cdots \vee y_k$, poichè uno degli y_i dev'essere necessariamente vero, ossia Y avrà un certo valore i nell'assegnazione totale;
- per ogni vincolo del CSP, si definirà una clausola per ogni assegnazione che lo violi:
  - $\neg(l_1 \wedge l_2 \wedge \cdots \wedge l_k)$ che equivale a $\neg l_1 \vee \neg l_2 \vee \cdots \vee \neg l_k$, ad esempio letterali per le assegnazioni alle Y_i non ammesse dal vincolo;

- per semplificare, si possono combinare le clausole; ad esempio, combinando $a \vee b \vee c$ con $a \vee b \vee \neg c$ si ha $a \vee b$.

Esempio 20. Considerando le variabili A e B con dominio $\{1, 2, 3, 4\}$
- per A, le variabili indicatrici saranno a_1, a_2, a_3 e a_4 con a_i vera se e solo se $A = i$, variabili disgiunte che coprono tutti i casi per cui, in tutto, 7 clausole:

$$\neg a_1 \vee \neg a_2, \neg a_1 \vee \neg a_3, \neg a_1 \vee \neg a_4, \neg a_2 \vee \neg a_3, \neg a_2 \vee \neg a_4, \neg a_3 \vee \neg a_4, a_1 \vee a_2 \vee a_3 \vee a_4$$

- analogamente per B;
- il vincolo $A < B$ implica che a_4 sika falsa, b_1 falsa e le coppie che lo violano siano parimenti false quindi un totale di 5 clausole: $\neg a_4, \neg b_1, \neg a_2 \vee \neg b_2, \neg a_3 \vee \neg b_3, \neg a_3 \vee \neg b_2$. Si noti che non serve $\neg a_1 \vee \neg b_1$ già implicata da $\neg b_1$.

L'algoritmo **DPLL Davis-Putnam-Logemann-Loveland** prevede:
- la riduzione dei domini e dei vincoli;
- la separazione dei domini;
- l'assegnazione di letterali *puri*.

L'algoritmo risulta efficiente con strutture dati indicizzate ad hoc.

4.3. Clausole Definite Proposizionali

Il linguaggio delle **clausole definite proposizionali** è un sottolinguaggio della logica proposizionale con la stessa semantica. Esso ammette una forma più specifica di proposizioni, le clausole con un unico letterale positivo della forma seguente:

$$a \vee \neg b_1 \vee \cdots \vee \neg b_m$$

Le *proposizioni atomiche* o *atomi* sono gli stessi della logica proposizionale. Valendo l'equiivalenza $A \vee \neg B \equiv A \leftarrow B$, una **clausola definita** proposizionale si può scrivere come segue:

$$h \leftarrow a_1 \wedge \cdots \wedge a_m$$

formula che si legge «h se a_1 e $\cdots$ e a_m». In tale formula, h si dice **testa** (*head*) e $a_1 \wedge \cdots \wedge a_m$ si dice **corpo** della clausola.
Una clausola definita è una **regola** se $m > 0$ (ossia ha un corpo che rappresenta una condizione), altrimenti è un **fatto** o **clausola atomica** quando $m = 0$ nel qual caso si può omettere $\leftarrow$.
Una **base di conoscenza** può essere specificata come insieme di clausole definite.
La clausola $h \leftarrow a_1 \wedge \cdots \wedge a_m$ è falsa nell'interpretazione I se $a_1, ..., a_m$ sono tutti veri in I ma h è falsa, altrimenti essa è vera in I.
La clausola definita è una forma ristretta di clausola nella quale compare esattamente un letterale positivo. Ad esempio, $a \leftarrow b \wedge c \wedge d$ è equivalente a $a \vee \neg b \vee \neg c \vee$

$\neg d$. Essa non può rappresentare una qualunque disgiunzione3 di atomi, nemmeno se tutti negati.

Esempio 21 *(Scheda impianto elettrico)* **.** Si consideri un ambiente intelligente per la simulazione (diagnostica) come quello nella figura sottostante:

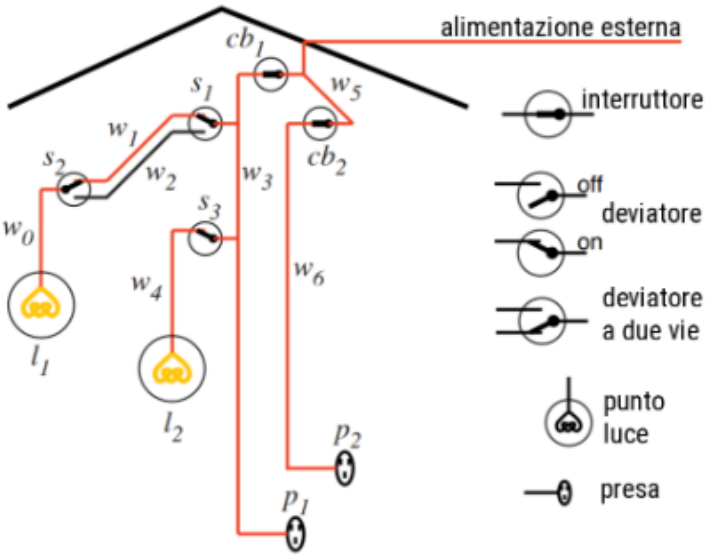


Figura 11: Impianto elettrico

- si definisce una *rappresentazione* dello stato di cavi (*wire*), prese (*outlet*), luci, deviatori (*switch*) e fusibili (*circuit breaker*), ecc.. I dettagli trascurabili non verranno modellati (ad es. colori, peso, lunghezze, altezze, ecc.);
- bisogna decidere il *livello di astrazione* per la rappresentazione: per *specialisti* (i voltaggi, le correnti, le frequenze, …) oppure comprensibile anche a non specialisti (in base al *senso comune*);
- poi si decide *cosa* rappresentare: proposizioni su accensione luci, funzionamento cavi, stato deviatori, ecc.;
- si definiscono *atomi* con significati precisi per il mondo da modellare: nomi descrittivi come *su_d2* per indicare che il deviatore *d2* è in posizione *su* oppure *alimentata_l1* che indica che il punto-luce *l1* è sotto tensione.
- si definisce anche la *conoscenza di fondo* (*background knowledge*, BK) che fornisce altri fatti veri riguardo il mondo, ed è rappresentata nel sistema attraverso altre clausole definite, spesso in forma semplice, come fatti:
  - *luce_l1.*
  - *luce_l2.*
  - *ok_l1.*
  - *ok_l2.*
  - *ok_f1.*
  - *ok_f2.*
  - *alimentazione_esterna.*

- si può anche considerare una parte del dominio e definire altre *regole*:
  - alimentata_l1 ← alimentato_c0.
    «(luce) « *l1* « alimentata se (cavo) « *c0* « sotto tensione»
  - alimentato_c0 ← alimentato_c1 ∧ su_d2.
  - alimentato_c0 ← alimentato_c2 ∧ giu_d2.
  - alimentato_c1 ← alimentato_c3 ∧ su_d1.
  - alimentato_c2 ← alimentato_c3 ∧ giu_d1.
  - alimentata_l2 ← alimentato_c4.
  - alimentato_c4 ← alimentato_c3 ∧ su_d3.
  - alimentata_p1 ← alimentato_c3.
  - alimentato_c3 ← alimentato_c5 ∧ ok_f1.
  - alimentata_p2 ← alimentato_c6.

  - alimentato_c6 ← alimentato_c5 ∧ ok_f2.
  - alimentato_c5 ← alimentazione_esterna.
  - accesa_l1 ← luce_l1 ∧ alimentata_l1 ∧ ok_l1.
  - accesa_l2 ← luce_l2 ∧ alimentata_l2 ∧ ok_l2.
- a run-time, si forniranno al sistema delle *osservazioni*, ad es. fatti sullo stato dei deviatori, come:
  - *giu_d1.*
  - *su_d2.*
  - *su_d3.*

4.3.1. Domande e Risposte

Si deve determinare la verità di fatti riguardanti il mondo ossia il dominio rappresentato:

- un utente può porre domande sulle conseguenze logiche della base di conoscenza fornita alla macchina;
- il sistema è in grado di dare risposte decidendo se date proposizioni seguano logicamente dalla base di conoscenza o meno;
- l'utente potrà comprendere il significato di tali risposte, se conosce la semantica degli atomi utilizzati.

A tale scopo si definisce una **query**, ossia una domanda utile a sapere se una data proposizione segua logicamente da una base di conoscenza, denotata con:

ask *b*

dove b è un atomo o una congiunzione di atomi il che corrisponde al corpo di una regola: $\leftarrow b_1 \wedge \cdots \wedge b_m$.
Le risposte possibili sono:

- yes se b è conseguenza logica, $KB \models b$;
- no altrimenti, $KB \not\models b$: ciò significa che b sia falso in un modello della KB, non necessariamente nell'interpretazione intesa, ma non è possibile determinarlo con la conoscenza disponibile.

4.3.2. Dimostrazioni

La relazione $\models$ definisce le risposte corrette secondo la *teoria dei modelli*. La **teoria ella dimostrazione**, *proof theory*, si occupa invece di come calcolare tqali risposte, ossia della dimostrazione delle conseguenza logiche. Si tratta di un problema di *deduzione*, una forma di *inferenza*.
Una **prova**, (*proof*), è una dimostrazione, derivabile anche *automaticamente*, del fatto che una proposizione seuga logicamente da un insieme di assiomi. Una proposizione dimostrabile si dice anche **teorema**.
Una **procedura di dimostrazione**, *proof procedure*, è un algoritmo, eventualmente non deterministico, utile a costruire prove. Data una procedura di dimostrazione, si denota con

$$KB \vdash g$$

la possibilità di produrre attraverso al procedura una prova del fatto che g sia conseguenza di KB.
Le proprietà più importanti delle procedure riguardano le relazioni tra prove e modelli:

- una procedura è **corretta** (*sound*) se e solo se ogni proposizione dimostrata dalla procedura è conseguenza logica di KB:

$$\text{se } KB \vdash g \text{ allora } KB \models g$$

- una procedura è **completa** se e solo se ogni conseguenza logica di KB può essere dimostrata dalla procedura:

$$\text{se } KB \models g \text{ allora } KB \vdash g$$

Esistono due tipi principali di procedure: le *bottom-up (BU)* e le *top-down (TD)*.

4.3.3. Procedura Bottom-Up

L'obiettivo di una procedura *bottom-up* è quello di derivare tutte le conseguenze logiche di una base di conoscenza. A tal fine si costruisce la prova gradualmente, basandosi su quanto è stato già dimostrato. Si procede per **concatenazione in avanti** (*forward chaining*) su clausole definite, derivando fati nuovi a partire da quanto è già stato dimostrato.

A tal fine si impiega una sola *regola di derivazione* che costituisce una generalizzazione della **regola d'inferenza** detta **modus ponens**

$$\frac{p \to q \quad p}{q}$$

Se $h \leftarrow a_1 \wedge \cdots \wedge a_m$ è in KB e ogni a_i è già stato provato, allora si potrà derivare h. Ogni fatto $(m = 0)$ in KB è un assioma quindi viene considerato immediatamente provato.
La seguente procedura trova C, l'*insieme delle conseguenze* di KB:

ALGORITMO 11 — *Procedura bottom-up*

```
1:  procedure Prove_DC_BU(KB)
2:  Input
3:  |  KB: insieme di clausole definite
4:  Output
5:  |  insieme di tutte le conseguenze logiche di KB
6:  Local
7:  |  C: insieme di atomi
8:  C ← ∅
9:  repeat
10: |  selezionare h ← a_1 ∧ ... ∧ a_m da KB
11: |      tale che ∀i : a_i ∈ C e h ∉ C
12: |  C ← C ∪ {h}
13: until nessuna clausola selezionabile
14: return C
```

La risposta restituita è `yes` ossia $KB \vdash g$ quando $g \in C$, se g atomica, ovvero quando $\{g_1, ..., g_k\} \subseteq C$ se g è una congiunzione di atomi $g_1 \wedge \cdots \wedge g_k$.

ESEMPIO 22. KB con gli assiomi:

- $a \leftarrow b \wedge c.$
- $b \leftarrow d \wedge e.$
- $b \leftarrow g \wedge e.$
- $c \leftarrow e.$
- $d.$
- $e.$
- $f \leftarrow a \wedge g.$

La traccia dei valori (insiemi) assegnati via via a C è la seguente:

- $\{\}$
- $\{d\}$
- $\{e, d\}$
- $\{c, e, d\}$
- $\{b, c, e, d\}$
- $\{a, b, c, e, d\}$

Da $C = \{a, b, c, e, d\}$ si può concludere che $KB \vdash a$, $KB \vdash b$, ecc.
Si noti che:

- non viene mai usata l'ultima regola di KB;
- non si riesce a provare né f né g, difatti c'è un modello della KB in cui f e g sono entrambe false.

La procedura bottom-up gode delle seguenti proprietà:

- *Punti Fissi*: C restituito è un **minimo punto fisso**:
  - ogni ulteriore applicazione della regola di derivazione non lo cambia;
  - detta I l'interpretazione in cui ogni atomo in C è vero e tutti gli altri sono falsi, I dev'essere un modello di KB;
  - I è un modello minimale di KB: ha il minor numero di proposizioni vere tra tutti i suoi modelli.
- *Correttezza*: ogni atomo in C è la conseguenza logica di KB
  - se $KB \vdash g$ allora $KB \models g$
- *Completezza*: tutte le conseguenze logiche sono derivabili
  - se $KB \models g$ allora g è vera in ogni modello di KB quindi anche in quello minimo, ovvero in C, per cui $KB \vdash g$.

4.3.4. Procedura Top-Down

Tale procedura effettua una ricerca *retrogada top-down* a partire da una query per determinare se questa segua logicamente dalle clausole di KB. Essa si basa su una diversa regola di ragionamento detta **risoluzione**:

$$\frac{l_1 \vee \cdots \vee a \vee \cdots \vee l_r \quad l'_1 \vee \cdots \vee \neg a \vee \cdots \vee l'_m}{l_1 \vee \cdots \vee l_r \vee l'_1 \vee \cdots \vee l'_m}$$

si noti che si applica alla forma generale delle clausole: basta individuare un atomo che compare positivamente in una delle due premesse e negativamente nell'altra.
In particolare, si usa la **risoluzione SLD** (Selezione di un atomo usando una strategia Lineare su clausole Definite):

$$\frac{a \leftarrow a_1 \wedge \cdots \wedge \underline{a_i} \wedge \cdots \wedge a_r \quad a_i \leftarrow b_1 \wedge \cdots \wedge b_m}{a \leftarrow a_1 \wedge \cdots \wedge \underline{b_1 \wedge \cdots \wedge b_m} \wedge \cdots \wedge a_r}$$

versione *proposizionale* di una forma più generale presentata in seguito.

Procedura Top-Down con Clausole di Risposta

Una **clausola di risposta** ha la forma

$$\text{yes} \leftarrow a_1 \wedge a_2 \wedge \cdots \wedge a_m$$

dove *yes* è un atomo *speciale*, vero se la risposta alla query è *yes*.
Data la query ask $q_1 \wedge \cdots \wedge q_m$, si parte dalla clausola di risposta iniziale

$$\text{yes} \leftarrow q_1 \wedge \cdots \wedge q_m$$

successivamente si ripete:

- la *selezione* di un atomo del corpo da dimostrare;
- la *scelta* di una clausola definita in KB avente come testa l'atomo selezionato per effettuare un passo di **risoluzione**; qualora non vi fossero altre clausole per dimostrare l'atomo selezionato, la dimostrazione *fallisce*.

Il **risolvente** della clausola di risposta e della clausola scelta sarà una nuova clausola di risposta:

$$\frac{\text{yes} \leftarrow \underline{a_1} \wedge a_2 \wedge \cdots \wedge a_m \quad a_1 \leftarrow b_1 \wedge \cdots \wedge b_p}{\text{yes} \leftarrow b_1 \wedge \cdots \wedge b_p \wedge a_2 \wedge \cdots \wedge a_m}$$

In sostanza l'atomo *goal* selezionato (a_1) viene sostituito dal corpo della clausola scelta.
In una dimostrazione di successo si raggiunge la **risposta**, ossia una clausola di risposta con corpo vuoto (tutti i goal sono stati dimostrati):

$$\text{yes} \leftarrow$$

Nella **derivazione SLD** per una query ask $q_1 \wedge \cdots \wedge q_k$ da KB si avrà una sequenza di clausole di risposta $\gamma_0, \gamma_1, ..., \gamma_n$ tali che:

- γ_0 è la clausola di risposta che ingloba la *query originaria* $\text{yes} \leftarrow q_1 \wedge \cdots \wedge q_k$;
- γ_i è il *risolvente* di γ_{i-1} con una clausola definita in KB;
- γ_n è la *risposta*.

Procedura Top-Down Alternativa

Si parte da un insieme G (*goal*) di *atomi da dimostrare*, quindi:

- si inizializza G con tutti gli atomi della query (*sotto-goal*) quindi G corrisponde a $\text{yes} \leftarrow \bigwedge_{g_i \in G} g_i$;
- selezionato a, la clausola scelta $a \leftarrow b_1 \wedge \cdots \wedge b_p$ indica che a può essere sostituito in G dai sotto-goal $b_1, ..., b_p$;
- si ha una terminazione con successo se $G = \emptyset$

ALGORITMO 12 — *Algoritmo Top-Down*

```
1: non-deterministic procedure Prove_DC_TD(KB, Query)
2: Input
3:     KB: insieme di clausole definite
4:     Query: insieme di atomi da dimostrare
5: Output
6:     yes se KB ⊨ Query, altrimenti fallisce (no)
```

```
Local
    G: insieme di atomi
G ← Query
repeat
    selezionare un atomo a ∈ G
    scegliere da KB una clausola definita a ← B con a come testa
    {se possibile, altrimenti il tentativo fallisce}
    G ← (G \ {a}) ∪ B
until G = ∅
return yes
```

Qualsiasi atomo di G può essere selzionato (tutti lo saranno). Se una seleziona non porta a terminare la prova, non serve tentarne selezionarne un altro, l'intera dimostrazione fallisce.
La procedura di *scelta* della clausola per la risoluzione è non deterministica. Se ci sono scelte che portano a G vuoto, l'algoritmo ha successo, altrimenti il tentativo fallisce.
Il corpo della clausola B e G sono considerati come insiemi di atomi. In alternativa G può essere costituito da una lista ordinata di atomi che possono occorrere più volte.

Esempio 23. Data la base di conoscenza con le seguenti clausole:

- $a \leftarrow b \wedge c.$
- $b \leftarrow d \wedge e.$
- $b \leftarrow g \wedge e.$
- $c \leftarrow e.$
- $d.$
- $e.$
- $f \leftarrow a \wedge g.$

e la query: *ask* a.
Si userà un insieme di goal G in luogo del corpo della clausola di risposta $yes \leftarrow G$, inizialmente gli atomi (della query) da dimostrare: $G = \{a\}$; si seleziona sempre l'atomo *più a sinistra* del corpo. I passi sono i seguenti:

risolventi γ_i	clausole da KB
$yes \leftarrow \underline{a}$	$a \leftarrow b \wedge c$
$yes \leftarrow \underline{b} \wedge c$	$b \leftarrow d \wedge e$
$yes \leftarrow \underline{d} \wedge e \wedge c$	d
$yes \leftarrow \underline{e} \wedge c$	e
$yes \leftarrow \underline{c}$	$c \leftarrow e$
$yes \leftarrow \underline{e}$	e

$$yes \leftarrow$$

Una sequenza *alternativa* di scelte in cui si sceglie la seconda clausola per b è la seguente:

- $yes \leftarrow a$
- $yes \leftarrow b \wedge c$
- $yes \leftarrow g \wedge e \wedge c$

ma selezionando g non ci sono regole da scegliere, per cui *questo tentativo fallisce*: occorre scegliere altre alternative, se ve ne sono.

Algoritmo TD su Grafo di Ricerca

Si può anche pensare a un algoritmo alternativo indotto dalla strategia di selezione basato su *grafo di ricerca* in cui:

- i *nodi* contengono clausole di risposta (bastano gli atomi nei loro corpi);
- i *vicini* di yes $\leftarrow \underline{a_1} \wedge \cdots \wedge a_m$ rappresentano tutte le possibili clausole di risposta ottenute risolvendo su a_1 e possono essere generati dinamicamente, uno per ogni clausola definita con testa a_1;
- i *nodi obiettivo* sono del tipo yes $\leftarrow$
- si può usare qualsiasi metodo di ricerca su grafo: basta trovare un percorso di successo perchè la query sia conseguenza logica della base di conoscenza; per ogni nodo, lo spazio viene determinato dalla query e dall'atomo selezionato.

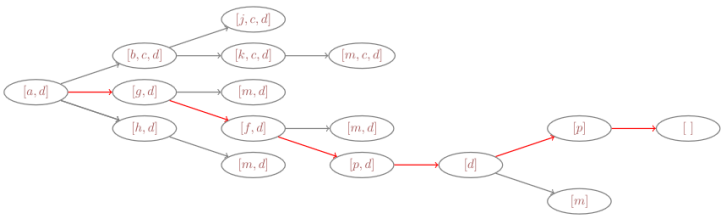


Figura 12: Albero di ricerca generato dall'algoritmo alternativo

Confronto fra procedure

Le dimostrazioni TD e BU sono interscambiabili: ciò può essere utile a provare la correttezza e la completezza della TD. La BU prova ogni atomo una sola volta mentre TD potrebbe provare lo stesso atomo più volte, ma si concentra su quelli rilevanti per la query. Con TD sono possibili cicli infiniti.

Esempio 24. Data la base di conoscenza

$$KB = \{g \leftarrow a. \quad a \leftarrow b. \quad b \leftarrow a. \quad g \leftarrow c. \quad c.$$

e la query ask g., si ha che:

- le uniche conseguenze logiche sono g e c;

- la procedura BU termina con il punto fisso $\{c, g\}$
- la procedura TD con DFS semplice potrebbe continuare indefinitamente a meno che non sia prevista la potatura dei cicli

La *strategia di selezione* dell'atomo per la risoluzione condiziona *efficienza* e *terminazione*:

- negli esempi precedenti: atomo più a sinistra, il che risulta problematico senza controllo sui cicli;
- una strategia migliore seleziona l'atomo che porti al fallimento più rapidamente;
- una strategia comune prevede l'ordinamento degli atomi per la selezione; ciò consente l'uso di un'euristica.

4.4. Interrogare l'utente

A run-time il sistema riceve **osservazioni** dagli utenti, da sensori o altre sorgenti. Esse sono rappresentate come *insiemi di atomi*. Le osservazioni interagiscono con la **conoscenza di fondo** (*background knowledge, BK*): possono esservi aggiunte o essere tenute separate dalla BK (base knowledge).
Le informazioni fornite dall'utente sono incomplete: è possibile che non sappia usare il vocabolario o non sappia giudicarne la rilevanza. Per questo servirebbero:

1. un'*ontologia* che specifichi il significato dei simboli
2. un'*interfaccia* che possa aiutare a fornire informazioni rilevanti.

Analogamente, i *sensori* possono essere *passivi*, quando raccolgono osservazioni dirette oppure *attivi*, se atti a raccogliere risposte a esplicite richieste di informazioni necessarie.
Per l'*acquisizione* si può pensare a un meccanismo **ask-the-user** integrato nella procedura di ragionamento: si considerano atomi dichiarati come **askable** per rappresentare fatti veri acquisibili a run-time dall'utente. Nella procedura TD, selezionato un atomo da dimostrare, si usa una clausola della base di conoscenza oppure, se è askable si chiede all'utente se sia vero o falso. Si noti che ciò vale solo per atomi rilevanti per la risposta, quindi si considerano tre diverse classi di atomi:

1. non askable;
2. askable senza risposta-utente: si può chiedere e memorizzare;
3. askable con risposta-utente memorizzata: usati senza chiedere di nuovo.

Si può adottare un meccanismo simile anche con procedure bottom-up, ma vanno evitate troppe richieste all'utente.

4.5. Debugging a Livello di Conoscenza

Data una semantica esplicita si possono realizare spiegazione e debugging a livello di conoscenza. Il **knowledge-level-debugging** consiste nel trovare errori nelle basi di conoscenza facendo riferimento solo al significato dei simboli e a quanto risulta vero nel mondo codificato e non ai passi di ragionamento.
I diversi tipi di errori possibili sono:

1. risposta *errata*: derivazione di atomo falso nell'interpretazione intesa;
2. risposta *mancante*: dimostrazione fallita per qualche atomo vero nell'interpretazione intesa;
3. *loop infinito*: solo con procedure TD, in base alla strategia di selezione. Ma si può evitare con meccanismi di potatura dei cicli.

4.5.1. Risposte Errate

Si tratta di risposte fornite dalla procedura di dimostrazione che sono *false* nell'interpretazione intesa, cioè casi di **falsi positivi**. Se la procedura è corretta allora nella dimostrazione sarà stata impiegata una clausola errata.
Per il *debugging dei falsi positivi*, si considerano l'atomo g dimostrato che, invece, dovrebbe essere falso nell'interpretazione intesa e la clausola usata per dimostrarlo $g \leftarrow a_1 \wedge \cdots \wedge a_k$. I casi possibili sono i seguenti:

- tutti gli a_i sono veri nell'interpretazione intesa, quindi la clausola $g \leftarrow a_1 \wedge \cdots \wedge a_k$ è errata;
- uno o più a_i dimostrati sono a loro volta falsi nell'interpretazione intesa, per cui si ha un analogo problema da risolvere.

L'algoritmo da usare può essere schematizzato come segue:

Algoritmo 13 — *Debug dei falsi positivi*

```
1: procedure Debug_false(g, KB)
2: Input
3:     KB: base di conoscenza
4:     g: atomo tale che KB ⊢ g ma falso nell'interpretazione intesa
5: Output
6:     clausola in KB che risulta falsa
7: trovare g ← a_1 ∧ ··· ∧ a_k ∈ KB usata per dimostrare g
8: chiedere all'utente se ogni a_i sia vero o meno
9: if l'utente indica un a_i falso then
10:     return Debug_false(a_i, KB)
11: else if l'utente risponde che tutti gli a_i sono veri then
```

12: | **return** $g \leftarrow a_1 \wedge \cdots \wedge a_k$

4.5.2. Risposte mancanti

In tal caso una risposta positiva attesa (ossia un goal vero nell'interpretazione intesa) non viene derivata dal sistema. Il *fallimento* dipende da un atomo g vero nell'interpretazione intesa ma tale che $KB \nvdash g$, si tratta, cioè, di un caso di **falso negativo**.

Tale atomo segnala il caso di una o più *clausole mancanti* tra gli assiomi della base di conoscenza e si può individuare con una procedura, come quella riportata, per trovare atomi cui manchino clausole utili alla loro dimostrazione:

Algoritmo 14 — *Debug dei falsi negativi*

1: **procedure** Debug_missing(g, KB)
2: **Input**
3: | KB: base di conoscenza
4: | g: atomo tale che $KB \nvdash g$ ma g vero nell'interpretazione intesa
5: **Output**
6: | atomo per il quale manchi una clausola utile
7: **if** $\exists g \leftarrow a_1 \wedge \cdots \wedge a_k \in KB$ con tutti gli a_i veri nell'interpretazione intesa **then**
8: | selezionare a_i che non può essere dimostrato
9: | **return** Debug_missing(a_i, KB)
10: **else**
11: | **return** g

4.6. Dimostrazione per Contraddizione

4.6.1. Clausole di Horn

Si intende estendere il ragionamento ammettendo regole che producano *contraddizioni*, ossia utili a specificare casi *impossibili*. Ciò ha anche lo scopo di implementare successvimente forme di *diagnostica*. A tale scopo il linguaggio delle clausole definite può essere esteso per esplicitare le contraddizioni. Definiti un atomo speciale, detto *false*, che codifica la contraddizione, ossia che è falso in tutte le interpretazioni, saranno amesse regole, dette **vincoli d'integrità**, rappresentate da clausole con l'atomo *false* come testa:

$$\text{false} \leftarrow a_1 \wedge \cdots \wedge a_k$$

Una **clausola di Horn** è una clausola definita o un vincolo d'integrità, ossia la sua testa è un atomo normale oppure l'atomo speciale *false*.
Da una base di conoscenza di clausole di Horn si possono derivare **negazioni** di atomi tramite la regola di ragionamento detta *modus tollens*:

$$\frac{p \rightarrow q \quad \neg q}{\neg p}$$

Esempio 25. Data la base di conoscenza KB_1:
1. false $\leftarrow a \wedge b$;
2. $a \leftarrow c$.
3. $b \leftarrow c$.

c è falso in tutti i modelli di KB_1. Pertanto si può scrivere $KB_1 \models \neg c$

Inoltre, valendo le seguenti equivalenze:

$$(\text{false} \leftarrow a_1 \wedge \cdots \wedge a_k) \equiv \neg(a_1 \wedge \cdots \wedge a_l) \equiv \neg a_1 \vee \cdots \vee \neg a_k$$

dimostrare che una *congiunzione di atomi* sia falsa in tutti i modelli di KB equivale a provare una **disgiunzione** di *negazione di atomi*.
Un insieme di clausole KB è **insoddisfacibile** se non ha modelli:

$$KB \models \text{false}$$

Un insieme di clausole KB è **inconsistente** rispetto a una procedura di dimostrazione $\vdash$ se consente di derivare false:

$$KB \vdash \text{false}$$

Se la procedura $\vdash$ è corretta e completa allora KB è inconsistente se e solo se è insoddisfacibile.
Una base di conoscenza di clausole definite è sempre soddisfacibile, per esempio basta considerare l'interpretazione che assegna il valore *true* a tutti gli atomi. Ciò non vale per le basi di clausole di Horn.

4.6.2. Assumibili e Conflitti

Le *contraddizioni* sono utili a individuare combinazioni di assunzioni incompatibili, ossia ipotesi fatte (per mancata conoscenza) che possono dimostrarsi false.
Si definisce **assumibile** un atomo che può essere assunto in una *dimostrazione per contraddizione*. In diagnostica, sarà utile a determinare una *disgiunzione di negazioni* di assumibili, provando la contraddizione a partire da una base di conoscenza e una combinazione di assumibili.
Si dice **conflitto** di KB l'insieme di assumibili $C = \{c_1, ..., c_r\}$ tale che $KB \cup C \models$ false. Si tratta di una **risposta** per una dimostrazione per contraddizione del

tipo: $KB \models \neg c_1 \vee \cdots \vee \neg c_r$. Un conflitto è **minimale** se nessun suo sottoinsieme costituisce, a sua volta, un conflitto.

Esempio 26. Data KB_2 e l'insieme di atomi assumibili $\{c, d, e, f, g, h\}$:

$$KB_2 = \begin{cases} \text{false} \leftarrow a \wedge b. \\ a \leftarrow c. \\ b \leftarrow d. \\ b \leftarrow e. \end{cases}$$

- $\{c, d\}$ e $\{c, e\}$ sono conflitti minimali;
- $\{c, d, e, h\}$ è un conflitto non minimale.

4.6.3. Diagnosi basata sulla consistenza

Lo scopo della diagnosi è quello di determinare possibili guasti in base a un modello del sistema, una base di conoscenza, e a osservazioni aggiuntive sul sistema stesso. In sostanza si fanno assunzioni sul funzionamento normale, considerando quindi assumibili in assenza di guasti. Si derivano i componenti *anormali*, ossia i conflitti, le anomalie nel sistema.

Esempio 27. (impianto elettrico, cont.) Si riconsideri la base di conoscenza *Scheda impianto elettrico* . Si aggiungono:

- *assunzioni* di normalità tramite *assumibili*: atomi ok^* per ogni componente (dev'essere in buono stato per funzionare);
- *osservazioni* fornite dall'utente che specificano le posizioni effettive (*su* / *giu*) dei deviatori, lo stato delle luci (*lit* / *spenta*),...;
- *vincoli d'integrità* utili a definire stati reciprocamente incompatibili come, ad esempio, che sia possibile un solo stato per i punti-luce:
  false ← *spenta_l1* ∧ *accesa_l1*.

KB risultante:

- *luce_l1*.
- *luce_l2*.
- *alimentazione_esterna*.
- *alimentata_l1* ← *alimentato_c0*.
- *alimentato_c0* ← *alimentato_c1* ∧ *su_d2* ∧ *ok_d2*.
- *alimentato_c0* ← *alimentato_c2* ∧ *giu_d2* ∧ *ok_d2*.
- *alimentato_c1* ← *alimentato_c3* ∧ *su_d1* ∧ *ok_d1*.
- *alimentato_c2* ← *alimentato_c3* ∧ *giu_d1* ∧ *ok_d1*.
- *alimentata_l2* ← *alimentato_c4*.
- *alimentato_c4* ← *alimentato_c3* ∧ *su_d3* ∧ *ok_d3*.
- *alimentata_*p_1 ← *alimentato_c3*.

- *alimentato_c3* ← *alimentato_c5* ∧ *ok_f1.*
- *alimentata_*p_2 ← *alimentato_c6.*
- *alimentato_c6* ← *alimentato_c5* ∧ *ok_f2.*
- *alimentato_c5* ← *alimentazione_esterna.*
- *accesa_l1* ← *luce_l1* ∧ *alimentata_l1* ∧ *ok_l1.*
- *accesa_l2* ← *luce_l2* ∧ *alimentata_l2* ∧ *ok_l2.*

vincoli d'integrità:

- *false* ← *spenta_l1* ∧ *accesa_l1.*
- *false* ← *spenta_l2* ∧ *accesa_l2.*

osservazioni:

- *su_d1.*
- *su_d2.*
- *su_d3.*
- *spenta_l1.*
- *spenta_l2.*

dichiarazioni:

- assumable *ok_f1,ok_f2,ok_d1,ok_d2,ok_d3,ok_l1,ok_l2.*

Tutto considerato, ci sono due conflitti minimali:

1. {*ok_f1,ok_d1,ok_d2,ok_l1*}
2. {*ok_f1,ok_d3,ok_l2*}

Pertanto:

1. $KB \models \neg$ *ok_f1* $\vee \neg$ *ok_d1* $\vee \neg$ *ok_d2* $\vee \neg$ *ok_l1*
   quindi almeno un componente fra *f1,*d_1*,*d_2*,l1* ha un guasto;
2. $KB \models \neg$ *ok_f1* $\vee \neg$ *ok_d3* $\vee \neg$ *ok_l2*
   ossia almeno uno tra *f1,*d_3*,l2* non funziona.

Dato l'insieme dei conflitti, l'utente può diagnosticare il problema: ciò è anche utile a farsi un'idea della numerosità dei possibili guasti.
Nella **diagnosi basata su consistenza** (CBD) dato un insieme di conflitti, cioè un insieme di assumbili con almeno un elemento in ogni conflitti, si costruisce una diagnosi che sarà **minimale** se nessun suo sotto-insieme è una diagnosi. Intuitivamente, una sola delle diagnosi minimali è quella giusta: qualla contenente solo conflitti falsi nell'interpretazione intesa.

4.6.4. Ragionamento con Assunzioni e Clausole di Horn

L'*obiettivo* è la ricerca dei *conflitti* in basi di conoscenza di clausole di Horn attraverso dimostrazioni per contraddizione dei due tipi considerati in precedenza.

Implementazione Bottom-Up

Si estende l'algoritmo bottom-up per clausole definite:

- le conclusioni sono *coppie* $\langle a, A\rangle$, con A insiemi di assumibili che implicano l'atomo a rispetto a KB;
- nell'inizializzazione, l'insieme di conclusioni sarà $C \leftarrow \{\langle a, \}a\}\rangle\} \mid a$ assumibile$\}$;
- si usano poi le clausole in KB per derivare nuove conclusioni: se $\exists h \leftarrow b_1 \wedge \cdots \wedge b_m \in KB$ tale che ogni $\langle b_i, A_i\rangle \in C$, allora si può aggiungere $\langle h, A_1 \cup ... \cup A_m$ a C

Algoritmo 15 — *Algoritmo bottom-up per la dimostrazione di conflitti*

```
1:  procedure Prove_conflict_BU(KB, As)
2:  Input
3:  |  KB: insieme di clausole di Horn
4:  |  As: insieme di atomi che si possono assumere veri
5:  Output
6:  |  insieme di conflitti
7:  Local
8:  |  C: insieme di coppie atomo/insieme di assumibili
9:  C ← {⟨a, {a}⟩ | a ∈ As}
10: repeat
11: |  selezionare la clausola h ← b_1 ∧ ... ∧ b_m ∈ KB tale che:
12: |      per ogni i, ⟨b_i, A_i⟩ ∈ C e ⟨h, A⟩ ∉ C, dove A = A_1 ∪ ... ∪ A_m
13: |  C ← C ∪ {⟨h, A⟩}
14: until nessun'altra selezione possibile
15: return {A | ⟨false, A⟩ ∈ C}
```

Implementazione Top-Down

L'implementazione top-down deriva dall'omologa per clausole definite, con due *differenze*:

- la query da provare è dall'atomo speciale *false*;
- nella dimostrazione, gli atomi assumibili non vanno dimostrati ma semplicemente raccolti via via assunti come veri.

Algoritmo 16 — *Algoritmo top-down per la dimostrazione di conflitti*

```
1: non-deterministic procedure Prove_conflict_TD(KB, As)
2: Input
3: |  KB: insieme di clausole di Horn
4: |  As: insieme di atomi che si possono assumere veri
5: Output
```

```
    | un conflitto
Local
    | G: insieme di atomi (che implicano false)
G ← {false}
repeat
    | selezionare un atomo a ∈ G tale che a ∉ As
    | scegliere una clausola a ← B di KB con a come testa
    | G ← (G \ {a}) ∪ B
until G ⊆ As
return G
```

Si osservi che l'algoritmo contiene diverse scelte *non deterministiche* in corrispondenza delle quali si ptoranni trovare diversi conflitti: in mancanza di scelte, l'algoritmo fallisce.

4.7. Assunzione di Conoscenza Completa

Nella semantica dele basi di dati quello che non è derivabile viene considerato falso: la conoscenza su un dominio viene considerata *completa*. In Logica la mancanza di conoscenza non consente di trarre conclusioni dal fallimento delle dimostrazioni.
Assumendo la **completezza della conoscenza**, le clausole con uno stesso atomo come testa coprono tutti e soli i casi in cui esso è vero. Tale assunzione si dice anche **closed-world (CWA)**. In base ad essa, se non si riesce a dimostrare che a sia vero, si conclude che è falso, ossia che la sua negzione $\neg a$ è vera. Si assume quindi che tutto quello che è rilevante sia stato asserito esplicitamente nella base di conoscenza.
In Logica normalmente si può assumere che una base di conosceza possa definire tutta la conoscenza su un dato diminio nella sua completezza, si parla pertanto di **open-world assumption (OWA)**. Quindi, non potendo trarre conclusioni dalla mancanza di conoscenza, la base di conoscenza di clausole definite non può avere $\neg a$ come conseguenza logica.

Completare la Base di Conoscenza

Per codificare l'assunzione di conosceza completa negli assiomi della base di conoscenza essa dev'essere trasfomrata ossia *completata*.
Dato l'atomo a, si considerino tutte le clausole della base di conoscenza che lo definiscono:

$$a \leftarrow b_1. \quad \ldots \quad a \leftarrow b_n.$$

dove b_i è una congiunzione di atomi oppure *true*. In mancanza di clausole per a si considera: $a \leftarrow$ false.
Tali clausole possono essere riassunte da un'unica proposizione:

$$a \leftarrow b_1 \vee \cdots \vee b_n$$

Se a è vero allora, per la CWA, dovrà necessariamente essere vero uno dei corpi b_i quindi si può aggiungere:

$$a \rightarrow b_1 \vee \cdots \vee b_n$$

Il significato delle clausole per l'atomo a sotto CWA è la congiunzione delle due proposizioni, ossia l'equivalenza:

$$a \Leftrightarrow b_1 \vee \cdots \vee b_n$$

detta *complemnto di Clark* delle clausole per a. Si noti che se non ci sono regole per a, il complemento sarà $a \Leftrightarrow$ false.
Il **complemento di Clark** della base di conoscenza ricomprende i complementi di tutti gli atomi della base.

Esempio 28. Si consideri la base di conoscenza precedente con le clausole:

- *giu_d1.*
- *su_d2.*
- *ok_f1.*
- *alimentata_l1* $\leftarrow$ *alimentato_c0.*
- *alimentato_c0* $\leftarrow$ *alimentato_c1* $\wedge$ *su_d2.*
- *alimentato_c0* $\leftarrow$ *alimentato_c2* $\wedge$ *giu_d2.*
- *alimentato_c1* $\leftarrow$ *alimentato_c3* $\wedge$ *su_d1.*
- *alimentato_c2* $\leftarrow$ *alimentato_c3* $\wedge$ *giu_d1.*
- *alimentato_c3* $\leftarrow$ *alimentazione_esterna* $\wedge$ *ok_f1.*
- *alimentazione_esterna.*

Si noti che non sono definite clausole per *su_d1* e *giu_d2*.
Completamento della base di conoscenza:

- *giu_d1* $\Leftrightarrow$ *true.*
- *su_d1* $\Leftrightarrow$ *false.*
- *su_d2* $\Leftrightarrow$ *true.*
- *giu_d2* $\Leftrightarrow$ *false.*
- *ok_f1* $\Leftrightarrow$ *true.*
- *alimentata_l1* $\Leftrightarrow$ *alimentato_c0.*
- *alimentato_c0* $\Leftrightarrow$ (*alimentato_c1* $\wedge$ *su_d2*) $\vee$ (*alimentato_c2* $\wedge$ *giu_d2*).
- *alimentato_c1* $\Leftrightarrow$ *alimentato_c3* $\wedge$ *su_d1.*
- *alimentato_c2* $\Leftrightarrow$ *alimentato_c3* $\wedge$ *giu_d1.*

- *alimentato_c3* $\Leftrightarrow$ *alimentazione_esterna* $\wedge$ *ok_f1.*
- *alimentazione_esterna* $\Leftrightarrow$ *true.*

Così, ad esempio, *su_d1* è falso, *alimentato_c1* è falso e *alimentato_c2* è vero.

Negazione come Fallimento

Un **letterale** è un atomo o la negazione di un atomo. Con il completamento si possono dimostrare negazioni di proposizioni, quindi la negazione può essere ammessa anche nel corpo delle clausole. Si considereranno clausole definite estese contenenti nel corpo letterali. La *negazione sotto assunzione di conoscenza completa*, detta anche **negation as failure (NaF)**, verrà denotata con $\sim a$ per distinguerla dalla negazione classica. Sotto CWA, il corpo g è una conseguenza di KB se e solo se $KB' \models g$, dove KB' è il completamento di KB. Si noti come $\sim a$ nel corpo di una clausola diventi $\neg a$ nel completamento.

Esempio 29. Si consideri l'assiomatizzazione vista in precedenza. Si semplifica la rappresentazione chiedendo all'utente di indicare solo i deviatori *su*, gli altri saranno *giu* per default. Analogamente, si potrebbe specificare che i fusibili sono *funzionanti* a meno che non risultino *rotti*.
Regole da aggiungere:

- *giu_d1* $\leftarrow\sim$ *su_d1.*
- *giu_d2* $\leftarrow\sim$ *su_d2.*
- *giu_d3* $\leftarrow\sim$ *su_d3.*
- *ok_f1* $\leftarrow\sim$ *rotto_f1.*
- *ok_f2* $\leftarrow\sim$ *rotto_f2.*

Per rappresentare lo stato della figura, l'utente specifica:

- *su_d2.*
- *su_d3.*

Il sistema può inferire che d_1 dev'essere in posizione *giu* e i due fusibili non sono guasti. Completamento:

- *giu_d1* $\Leftrightarrow \neg$ *su_d1.*
- *giu_d2* $\Leftrightarrow \neg$ *su_d2.*
- *giu_d3* $\Leftrightarrow \neg$ *su_d3.*
- *ok_interruttore1* $\Leftrightarrow \neg$ *rotto_interruttore1.*
- *ok_interruttore2* $\Leftrightarrow \neg$ *rotto_interruttore2.*
- *su_d1* $\Leftrightarrow$ *false.*
- *su_d2* $\Leftrightarrow$ *true.*
- *su_d3* $\Leftrightarrow$ *true.*
- *rotto_interruttore1* $\Leftrightarrow$ *false.*
- *rotto_interruttore2* $\Leftrightarrow$ *false.*

Con la NAF, le basi di conoscenza non acicliche diventa problematiche dal punto di vista semantico. Ad esempio, data la base non aciclica:

- $a \leftarrow\sim b$
- $b \leftarrow\sim a$

il completamento equivalente a $a \Leftrightarrow \neg b$, indica che a e b hanno valori di verità opposti: solo uno può essere vero.
Un altro esempio di base di conoscenza non aciclica è una con l'assioma:

- $a \leftarrow\sim a$

con completamento:

- $a \Leftrightarrow \neg a$

che risulta *inconsistente* dal punto di vista logico. Il completamento di una base aciclica è sempre consistente e prevede un solo valore di verità per ogni atomo.

4.7.1. Ragionamento Non Monotono

In una logica **monotona** ogni proposizione derivabile da una base di conoscenza rimane derivabile dopo l'aggiunta di altre proposizioni: aggiungendo assioni non si riduce l'insieme delle proposizioni derivabili. In una *Logica non monotona* alcune conclusioni possono essere invalidate con l'aggiunta di altri assiomi.

Default ed Eccezioni

Una logica non monotona è utile a rappresentare casi predefiniti. Si dice **default** una regola che resta valida fintantochè non si verifichi un'eccezione. Ad esempio per asserire che normalmente b è vera se c è vera si può definire la regola:

$$b \leftarrow c \wedge \sim ab_a$$

con ab_a che indica l'anormalità rispetto a qualche aspetto di a. Dato c si può derivare b, a meno che non venga asserito ab_a, in quanto l'aggiunta di ab_a inibisce la possibilità di concludere b. Le regole che implicano ab_a possono inibire il default, sotto le condizoni del corpo della regola. Ad esempio, nella maggior parte dei casi gli uccelli (c) sono volatili (b), ma i pinguini fanno eccezione (ab_a).

4.7.2. Procedure di Dimostrazione per la NAF

Procedura Bottom-Up + NAF

La procedura BU per clausole definite va modificata: quando p fallisce va aggiunto il letterale $\sim p$ all'insieme C delle conseguenze derivate.
Si ha una definzione ricorsiva del **fallimento**: p fallisce se fallisce il corpo di ogni clausola che abbia p coem testa; un corpo fallisce se almeno uno dei suoi letterali fallisce; un atomo b_i fallisce se si può derivare $\sim b_i$ e la negazione $\sim b_i$ fallisce se si può derivare b_i.

Algoritmo 17 — *Algoritmo bottom-up di dimostrazione con NAF*

```
1: procedure Prove_NAF_BU(KB)
```

Input
  KB: insieme di clausole che possono includere NAF
Output
  insieme di letterali che seguono dal completamento di KB
Local
  C insieme di letterali
  $C \leftarrow \{\}$
repeat
  either
    selezionare $(h \leftarrow b_1 \wedge \cdots \wedge b_m) \in KB$ tale che: $h \notin C$ e $\forall i : b_i \in C$
    $C \leftarrow C \cup \{h\}$
  or
    selezionare h con $\sim h \notin C$ tale che:
      $\forall (h \leftarrow b_1 \wedge \cdots \wedge b_m) \in KB : \exists i : \sim b_i \in C$ oppure $b_i = \sim g$ e $g \in C$
    $C \leftarrow C \cup \{\sim h\}$
until non ci sono altre selezioni possibili

Esempio 30. Si considerino le clausole:

1. $p \leftarrow q \wedge \sim r.$
2. $p \leftarrow s.$
3. $q \leftarrow \sim s.$

1. $r \leftarrow \sim t.$
2. $t.$
3. $s \leftarrow w.$

Una possibile sequenza di letterali aggiunti a C è la seguente:

- $C = \{t\}$ — 5. (fatto)
- $C = \{t, \sim r\}$ — $t \in C$ e 4.
- $C = \{t, \sim r, \sim w\}$ — non ci sono clausole per w
- $C = \{t, \sim r, \sim w, \sim s\}$ — $\sim w \in C$ e 6.
- $C = \{t, \sim r, \sim w, \sim s, q\}$ — $\sim s \in C$ e 3.
- $C = \{t, \sim r, \sim w, \sim s, q, p\}$ — $q \in C$, $\sim r \in C$ e 1.

Procedura Top-Down + NAF

La procedura TD è analoga a quella delle clausole definite, ma procede per NAF. È non-deterministca e, come tale, si può implementare come ricerca di scelte di successo: selezionato un atomo $\sim a$, parte una nuova dimostrazione per a; se questa fallisce allora $\sim a$ ha successo altrimenti l'algoritmo fallisce: deve tentare altre scelte se possibile.

Algoritmo 18 — *Algoritmo top-down di dimostrazione con NAF*

```
1: non-deterministic procedure Prove_NAF_TD(KB, Query)
2: Input
3:     KB: insieme di clausole che includono la NAF
4:     Query: insieme di letterali da dimostrare
5: Output
6:     yes se Query segue dal completamento di KB, "fail" altrimenti
7: Local
8:     G: insieme di letterali
9: G ← Query
10: repeat
11:     selezionare il letterale l ∈ G
12:     if l =∼ a then
13:         if Prove_NAF_TD(KB, a) = 'fail' then
14:             G ← G \ {l}
15:         else
16:             return "fail"
17:     else
18:         scegliere una clausola l ← B ∈ KB
19:         G ← G \ {l} ∪ B
20: until G = ∅
21: return yes
```

Esempio 31. Data la base di conoscenza nell'Esempio 30 la query `ask` p:

- $G = \{p\}$ – inizializzazione
- $G = \{q, \sim r\}$ – sostituzione con il corpo di 1.;
- $G = \{\sim s, \sim r\}$ – sostituendo q con il corpo della 3.;
- $G = \{\sim r\}$ – selezionato ed eliminato $\sim s$ la dim. di s *fallisce*;
- $G = \{\}$ – selezionato $\sim r$, la dim. di r *fallisce* poiché, usando la 4., si deve provare il sotto-goal $\sim t$, si tenta quindi di dimostrare t; data 5., tale dim. ha *successo* immediato, e quella di $\sim t$ *fallisce*. non ci sono altre regole per r. da cui il successo di $\sim r$ che può essere eliminato.

Essendo G vuoto l'output sarà `yes`.

Quanto detto vale solo enl caso di fallimento finito: nessuna conclusione è possibile in caso di divergenza. Ad esempio, data una base di conoscenza con la sola regola $p \leftarrow p$ e la query ask p, l'algoritmo non converge: il completamento è $p \Leftrightarrow p$ ma, pur potendosi accertare l'indimostrabilità di p, una procedura corretta non può concludere $\sim p$ poichè non segue logicamente dal completamento.

4.8. Abduzione

L'**abduzione** è una forma di ragionamento nella quale si fanno ipotesi, ossia assunzioni, al fine di spiegare osservazioni. Le ipotesi da farsi possono implicare le osservazioni fatte: se verificate, esse giustificano quanto osservato ma non si devono introdurre *contraddizioni* poichè queste giustificherebbero qualsiasi conclusione.
L'abduzione si differenzia dalla deduzione la quale mira alle conseguenze logiche di un set di assiomi e dall'induzione che comporta l'inferenza di relazioni generali da casi particolari.

Scenari e Spiegazioni

Possiamo schematizzare questa forma di ragionamento come segue. I dati sono i seguenti:

- una base KB di Horn;
- un insieme A di atomi utili a costruire ipotesi, detti assumibili o anche abducibili,

dobbiamo trovare delle spiegazioni per delle osservazioni.
Si dice **scenario** di $\langle KB, A\rangle$ un insieme di abducibili $H \subseteq A$ tale che $KB \cup H$ sia soddisfacibile. In tal caso deve esistere un modello in cui tutti gli elementi di KB e di H sono veri, quindi nessun sottoinsieme di H deve essere un conflitto per KB.
Una **spiegazione** della proposizione g da $\langle KB, A\rangle$ è uno scenario $H \subseteq A$ che, assieme a KB, abbia g come conseguenza, quindi:

- $KB \cup H \models g$ e
- $KB \cup H \not\models \text{false}$

Essa è **minimale** se H non contiene altre spiegazioni: per ogni altra H' di g da $\langle KB, A\rangle$ deve risultare $H' \not\subset H$.

> **Esempio 32.** Si consideri una base di conoscenza con assumibili per la *diagnosi medica*:
>
> - *bronchite* ← *influenza.*
> - *bronchite* ← *fumatore.*
> - *tosse* ← *bronchite.*
> - *affanno* ← *bronchite.*
> - *febbre* ← *influenza.*
> - *febbre* ← *infezione.*
> - *mal_di_gola* ← *influenza.*
> - *false* ← *fumatore* ∧ *non_fumatore.*
> - assumable *fumatore, non_fumatore, influenza, infezione.*
>
> Se si osserva *affanno* (respiro affannoso), due spiegazioni minime, {*influenza*} e {*fumatore*}, che implicano *bronchite* e *tosse* (tosse).
> Se si osservano *affanno* ∧ *febbre*, spiegazioni minimali: {*influenza*} e {*fumatore,infezione*}.
> Se si osservano *affanno* ∧ *non_fumatore*, spiegazione minimale: {*influen-*

za,non_fumatore}. Anche {*fumatore*} potrebbe spiegare *affanno*, ma il vincolo la rende inconsistente rispetto a *non_fumatore* osservato.

Esempio 33. Si consideri la base di conoscenza:

- *allarme* ← *manomissione.*
- *allarme* ← *incendio.*
- *fumo* ← *incendio.*

Se si osserva *allarme*, spiegazioni minimali sono: {*manomissione*} e {*incendio*}. Se si osservano *allarme* ∧ *fumo*, la spiegazione minimale è solo {*incendio*}. Difatti avendo osservato *fumo* non c'è bisogno di ipotizzare *manomissione* per spiegare *allarme*, è già spiegato da *incendio.*

Diagnosi Abduttiva

La *diagnosi abduttiva (AbD)* formula iptoesi su problemi riscontrati e funzionamento normale.
Confrontandola con quella basata su consistenza (CbD):

- al livello di *rappresentazione*, la CbD mira solo al comportamento normale quindi le ipotesi saranno assunzioni sul comportamento normale, mentre con la AbD si modella anche il comportamento anomalo, si devono prevedere assumibili anche per ciascun guasto;
- per quanto riguarda le osservazioni, la CbD le aggiunge alla base di conoscenza per poi dimostrare *false* mentre la AbD le considera come goal da spiegare attraverso ipotesi;
- in termini di qualità, la AbD risulta più dettagliata,le diagnosi sono migliori: si distinguono base di conoscenza e assunzioni consentono di dimostrare le osservazioni, anche per casi in cui non sia definibile la normalità.

Ragionamento Abduttivo: Procedure

Il ragionamento abduttivo è basato su assunzioni e si realizza mediante le procedure su clausole di Horn:

- con la procedura BU si calcolano spiegazioni minimali per ogni atomo, essa è ulteriormente specialzizabile con il pruning delle spiegazioni dominate;
- con la procedura TD si trovano spiegazioni di un g generando i conflitti, ma mirando a dimostrare g anzichè *false*: la spiegazione minimale di g sarà un insieme minimale di assumibili raccolti nella prova.